Markov Decision Processes and Multi-Armed Bandits: Overview and Worked Examples

Gregory Zancewicz*

February 3, 2026

Abstract

This paper provides a self-contained overview of six core model classes in Markov-based sequential decision-making: Markov processes, Markov decision processes (MDPs), partially observable MDPs (POMDPs), multi-armed bandits, contextual bandits, and restless bandits. The models are arranged in a progression of increasing generality, beginning with passive stochastic dynamics and culminating in problems where multiple evolving subsystems must be managed under combinatorial constraints. For each model class we present a formal definition, the principal theoretical results—including the Bellman equation, Bayesian belief updates, regret bounds, and the Whittle index—and one or more fully worked numerical examples with accompanying Python code. Bandit strategies covered include ε-greedy, UCB1, Thompson Sampling, the Gittins index, successive elimination, KL-UCB, EXP3, LinUCB, and EXP4. We also briefly survey hidden Markov models, Markov games, constrained MDPs, semi-Markov decision processes, and model-free reinforcement learning. A glossary of all technical terms is provided. The paper is intended as an accessible yet rigorous entry point for researchers and practitioners new to the field.

*https://www.linkedin.com/in/zancewicz/

Contents

1 Introduction **6**

2 Markov Processes **7**

2.1 Notation 7

2.2 Definitions and Notation 7

2.3 Two-State Example 8

2.4 Summary 9

2.4.1 Key Equations 9

2.4.2 Relevant Applications 9

3 Markov Decision Processes **10**

3.1 Notation 10

3.2 Formal Definition 10

3.3 Policies 11

3.4 The Value Function and Bellman's Equation 11

3.5 Solution Methods 12

3.6 Examples 12

3.6.1 Example 1: Basic Binary MDP 12

3.6.2 Example 2: Simple Asset Maintenance MDP 14

3.7 Summary 16

3.7.1 Key Equations 17

3.7.2 Relevant Applications 17

4 Partially Observable Markov Decision Processes **18**

4.1 Notation 18

4.2 Formal Definition 18
4.3 Belief States 19
4.4 The Value Function and Bellman's Equation 19
4.5 Example: The Tiger Problem 20
4.6 Summary 20
4.6.1 Key Equations 21
4.6.2 Relevant Applications 22

5 Multi-Armed Bandit Problems **22**
5.1 Notation 23
5.2 Formal Definition 23
5.3 Strategies 24
5.3.1 ε-Greedy 24
5.3.2 Upper Confidence Bound (UCB1) 24
5.3.3 Thompson Sampling 25
5.3.4 The Gittins Index 25
5.3.5 Successive Elimination 25
5.3.6 KL-UCB 26
5.3.7 EXP3 26
5.4 Regret Bounds 26
5.5 Examples 27
5.5.1 ε-Greedy 27
5.5.2 UCB1 28
5.5.3 Thompson Sampling 29
5.5.4 The Gittins Index 29

5.6 Summary 29
5.6.1 Key Equations 30
5.6.2 Relevant Applications 30

6 Contextual Bandit Problems **31**
6.1 Notation 32
6.2 Formal Definition 32
6.3 LinUCB: Linear Upper Confidence Bound 33
6.4 Thompson Sampling for Contextual Bandits 34
6.5 EXP4: Adversarial Contextual Bandits 35
6.6 Example: News Article Recommendation 35
6.7 Summary 37
6.7.1 Key Equations 37
6.7.2 Relevant Applications 37

7 Restless Bandit Problems **38**
7.1 Notation 38
7.2 Formal Definition 38
7.3 Intractability 39
7.4 Whittle's Relaxation and the Whittle Index 39
7.5 Example: Machine Maintenance 40
7.6 Summary 43
7.6.1 Key Equations 43
7.6.2 Relevant Applications 43

8 Beyond the Scope of This Paper **44**

8.1 Hidden Markov Models 44

8.2 Markov Games 45

8.3 Constrained MDPs 45

8.4 Semi-Markov Decision Processes 46

8.5 Model-Free Reinforcement Learning 46

9 Conclusion **47**

1 Introduction

Many problems in engineering, operations research, economics, and artificial intelligence share a common structure: an agent must make a sequence of decisions under uncertainty, where each decision influences future states of the world and the rewards that follow. The mathematical framework for such problems rests on the theory of Markov processes and their controlled extensions, a body of work spanning over a century—from Markov's original study of dependent random variables [1] through Bellman's dynamic programming [2] to modern bandit algorithms deployed in clinical trials, online advertising, and autonomous systems.

This paper provides a self-contained overview of six core model classes arranged in a progression of increasing generality:

1. **Markov Processes** (Section 2). A system evolves randomly among a finite set of states according to fixed transition probabilities, with no external control. The transition matrix and stationary distribution are the central objects.

2. **Markov Decision Processes** (Section 3). An agent chooses actions that influence both state transitions and rewards. The Bellman equation characterizes the optimal policy, and solution methods—value iteration, policy iteration, and linear programming—provide constructive algorithms.

3. **Partially Observable MDPs** (Section 4). The agent can no longer observe the state directly and must maintain a belief distribution updated via Bayes' rule. The POMDP reduces to a continuous-state MDP over the belief simplex, at the cost of greatly increased computational complexity.

4. **Multi-Armed Bandits** (Section 5). The exploration–exploitation tradeoff is isolated in its purest form: a single-state problem where the agent must learn which arm is best by trying them. We cover ε-greedy, UCB1, Thompson Sampling, the Gittins index, successive elimination, KL-UCB, and EXP3, together with the Lai–Robbins regret lower bound.

5. **Contextual Bandits** (Section 6). Side information (a context vector) is observed before each arm selection, and reward distributions depend on the context. The agent must learn a mapping from contexts to optimal arms. We cover LinUCB, Thompson Sampling for linear models, and EXP4 for adversarial settings.

6. **Restless Bandits** (Section 7). The classical bandit assumption that unplayed arms are frozen is removed: every arm evolves at every time step. The Whittle index policy provides a tractable heuristic for this PSPACE-hard problem.

Section 8 briefly surveys several related model classes—hidden Markov models, Markov games, constrained MDPs, semi-Markov decision processes, and model-free reinforcement

learning—that lie beyond the scope of this paper but occupy adjacent regions of the same theoretical landscape.

Each section follows a uniform structure: a notation table listing all symbols used in that section, a formal definition, the key theoretical results, one or more worked examples with explicit numerical computation, and a summary containing the key equations and relevant applications. Where appropriate, Python code is provided to verify analytical solutions via value iteration or simulation. The paper concludes with a glossary of all technical terms.

This paper is pedagogical in intent; it presents no new results but aims to collect, unify, and illustrate the core ideas in a single self-contained reference. The intended audience is researchers and practitioners who seek a concise, rigorous, yet accessible entry point into Markov-based sequential decision-making. Familiarity with linear algebra and basic probability is assumed; no prior exposure to dynamic programming or bandit theory is required.

2 Markov Processes

A **Markov process** (or Markov chain) is a stochastic process satisfying the *Markov property*: the conditional probability of future states depends only on the present state, not on the sequence of states that preceded it. This idea was first formalized by A. A. Markov in his study of dependent random variables [1].

2.1 Notation

Symbol	Description
$\mathcal{S} = \{s_1, \ldots, s_n\}$	Finite set of states
t	Discrete time index, $t = 0, 1, 2, \ldots$
X_t	State occupied at time t
p_{ij}	Transition probability from state s_i to state s_j
n	Number of states, $\lvert\mathcal{S}\rvert$
$\mathbf{P}$	Transition matrix, $[\mathbf{P}]_{ij} = p_{ij}$
$\boldsymbol{\mu}_t$	Row vector distribution over states at time t
$\boldsymbol{\mu}^*$	Stationary distribution, $\boldsymbol{\mu}^* = \boldsymbol{\mu}^*\mathbf{P}$
α	Transition probability $s_1 \to s_2$ (two-state example)
β	Transition probability $s_2 \to s_1$ (two-state example)

2.2 Definitions and Notation

Let $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ denote a finite set of **states**. Each state is simply an element of this set; it may represent a concrete condition (e.g., "sunny" or "rainy"), a configuration of a

system, or, in some applications, a random variable—but need not be in general. A discrete-time Markov chain is a process that, at each time step $t = 0, 1, 2, \ldots$, occupies some state $X_t \in \mathcal{S}$ and transitions to the next state. The defining assumption—the *Markov property*—is that the probability of the next state depends only on the current state, not on how the process arrived there:

$$P(X_{t+1} = s_j \mid X_t = s_i, X_{t-1}, \ldots, X_0) = P(X_{t+1} = s_j \mid X_t = s_i) \triangleq p_{ij}. \tag{1}$$

That is, conditioning on the full history (left side) yields the same probability as conditioning on the present state alone (right side). We denote this common value p_{ij}, the **transition probability** from state s_i to state s_j. The transition probabilities satisfy

$$p_{ij} \geq 0 \quad \text{and} \quad \sum_{j=1}^{n} p_{ij} = 1 \quad \text{for all } i. \tag{2}$$

These probabilities are collected in the **transition matrix**

$$\mathbf{P} = \begin{pmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ p_{21} & p_{22} & \cdots & p_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{pmatrix}, \tag{3}$$

where each row sums to one. The distribution over states at time t is a row vector $\boldsymbol{\mu}_t$, and the evolution of the chain is given by $\boldsymbol{\mu}_{t+1} = \boldsymbol{\mu}_t \mathbf{P}$.

A distribution $\boldsymbol{\mu}^*$ is called a **stationary distribution** if $\boldsymbol{\mu}^* = \boldsymbol{\mu}^* \mathbf{P}$.

2.3 Two-State Example

Consider a Markov chain with two states, $\mathcal{S} = \{s_1, s_2\}$, and transition matrix

$$\mathbf{P} = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix}, \tag{4}$$

where $\alpha = P(X_{t+1} = s_2 \mid X_t = s_1)$ and $\beta = P(X_{t+1} = s_1 \mid X_t = s_2)$. The corresponding state diagram is shown in Figure 1.

Provided $\alpha, \beta > 0$, this chain is ergodic[1] and admits the stationary[2] distribution

$$\boldsymbol{\mu}^* = \left(\frac{\beta}{\alpha+\beta}, \frac{\alpha}{\alpha+\beta} \right). \tag{5}$$

[1] A Markov chain is *ergodic* if it is irreducible (every state is reachable from every other state) and aperiodic (the chain does not cycle among subsets of states with a fixed period). An ergodic chain converges to a unique stationary distribution regardless of the initial state.

[2] A distribution that, once reached, remains unchanged under further transitions of the chain.

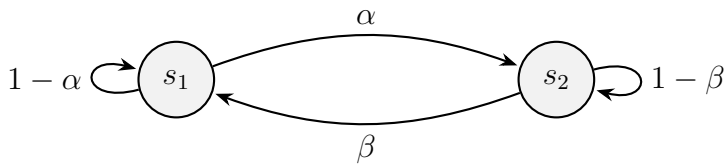

Figure 1: A two-state Markov chain with transition probabilities α and β.

2.4 Summary

A Markov process models a system that transitions randomly among a finite set of states, with the defining property that the future depends only on the present state—not on the path taken to reach it. The transition matrix $\mathbf{P}$ fully characterizes the dynamics, and the distribution over states evolves by left-multiplication: $\boldsymbol{\mu}_{t+1} = \boldsymbol{\mu}_t \mathbf{P}$. Under mild conditions (ergodicity), the chain converges to a unique stationary distribution regardless of the starting state. Markov processes provide the probabilistic foundation for all models in the subsequent sections: MDPs add actions and rewards, POMDPs add partial observability, and bandit models isolate the exploration–exploitation tradeoff.

2.4.1 Key Equations

Markov property:

$$P(X_{t+1} = s_j \mid X_t = s_i, X_{t-1}, \ldots, X_0) = P(X_{t+1} = s_j \mid X_t = s_i) \triangleq p_{ij}.$$

Distribution evolution:

$$\boldsymbol{\mu}_{t+1} = \boldsymbol{\mu}_t \mathbf{P}.$$

Stationary distribution:

$$\boldsymbol{\mu}^* = \boldsymbol{\mu}^* \mathbf{P}.$$

Stationary distribution (two-state chain):

$$\boldsymbol{\mu}^* = \left(\frac{\beta}{\alpha+\beta}, \frac{\alpha}{\alpha+\beta} \right).$$

2.4.2 Relevant Applications

- Weather modeling, where atmospheric conditions transition among discrete states (e.g., sunny, cloudy, rainy) with estimated probabilities.
- Queueing systems, where the number of customers in a queue evolves as a Markov chain governed by arrival and service rates.

- PageRank and web link analysis, where the stationary distribution of a random walk on the web graph determines page importance.
- Population genetics, where allele frequencies evolve across generations via random drift modeled as a Markov chain.
- Speech and language modeling, where sequences of phonemes or words are generated by hidden or observable Markov models.

3 Markov Decision Processes

A Markov decision process extends the Markov chain of Section 2 by introducing *actions* and *rewards*. Where a Markov chain evolves passively according to fixed transition probabilities, an MDP allows an agent to *choose* an action at each time step, thereby influencing both the transition to the next state and the reward received. This formulation originates with Bellman [2], who studied the asymptotic behavior of nonlinear recurrence relations arising from dynamic programming.

3.1 Notation

Symbol	**Description**
$\mathcal{A} = \{a_1, \ldots, a_m\}$	Finite set of actions
$p_{ij}(a)$	Transition probability from s_i to s_j under action a
$r(s_i, a)$	Immediate reward for taking action a in state s_i
γ	Discount factor, $\gamma \in [0, 1)$
π	Policy, a mapping $\pi : \mathcal{S} \to \mathcal{A}$
$V : \mathcal{S} \to \mathbb{R}$	Value function
$V^\pi(s_i)$	Value of state s_i under policy π
$V^*(s_i)$	Optimal value of state s_i
π^*	Optimal policy
$f_N(s_i)$	Optimal return over N remaining stages (Bellman's notation)
$b(s_i, a)$	Immediate return (Bellman's notation)
$a_{ij}(a)$	Transition coefficients (Bellman's notation)

3.2 Formal Definition

A Markov decision process is defined by the tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$:

- $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$: a finite set of **states**, as in Section 2.

- $\mathcal{A} = \{a_1, a_2, \ldots, a_m\}$: a finite set of **actions** (or decisions) available to the agent.
- $p_{ij}(a)$: the **transition probability** of moving from state s_i to state s_j when action a is taken. For each state s_i and action a,

$$p_{ij}(a) \geq 0 \quad \text{and} \quad \sum_{j=1}^{n} p_{ij}(a) = 1. \tag{6}$$

  Note that the transition probabilities now depend on the chosen action—this is the key departure from the passive Markov chain, where p_{ij} was fixed.
- $r(s_i, a)$: the **immediate reward** (or return) received when action a is taken in state s_i.
- $\gamma \in [0, 1)$: a **discount factor** that weights future rewards relative to immediate ones.

3.3 Policies

A **policy** $\pi : \mathcal{S} \to \mathcal{A}$ is a rule that specifies which action to take in each state. More generally, a policy may be *stochastic*, specifying a distribution $\pi(a \mid s)$ over actions in each state; however, for finite-state discounted MDPs a deterministic stationary policy always suffices for optimality [26]. Under a fixed deterministic policy π, the MDP reduces to an ordinary Markov chain with transition probabilities $p_{ij}(\pi(s_i))$.

3.4 The Value Function and Bellman's Equation

Following Bellman's approach, we seek a **value function** $V : \mathcal{S} \to \mathbb{R}$ that assigns to each state the maximum total discounted reward obtainable from that state onward. Formally, for a policy π the value of state s_i is

$$V^{\pi}(s_i) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \, r(X_t, \pi(X_t)) \;\middle|\; X_0 = s_i\right], \tag{7}$$

where $X_t \in \mathcal{S}$ is the state at time t and the expectation is over the stochastic transitions. The **optimal value function** is then $V^*(s_i) = \max_{\pi} V^{\pi}(s_i)$. Bellman showed that V^* must satisfy the recurrence relation

$$V^*(s_i) = \max_{a \in \mathcal{A}} \left[r(s_i, a) + \gamma \sum_{j=1}^{n} p_{ij}(a) \, V^*(s_j) \right], \quad i = 1, 2, \ldots, n. \tag{8}$$

This is the **Bellman equation**. It expresses a principle of optimality: the value of a state equals the best immediate reward plus the discounted expected value of the successor state, maximized over all available actions.

In Bellman's original notation [2], this appears as the nonlinear recurrence

$$f_N(s_i) = \max_a \left[b(s_i, a) + \sum_{j=1}^{n} a_{ij}(a)\, f_{N-1}(s_j) \right], \tag{9}$$

where $f_N(s_i)$ is the optimal return over N remaining stages, $b(s_i, a)$ is the immediate return[3], and $a_{ij}(a)$ are the transition coefficients. Our formulation above corresponds to the infinite-horizon, discounted case obtained by letting $N \to \infty$ with discount factor γ.

The optimal policy π^* is then the one that achieves the maximum in the Bellman equation at every state:

$$\pi^*(s_i) = \arg\max_{a \in \mathcal{A}} \left[r(s_i, a) + \gamma \sum_{j=1}^{n} p_{ij}(a)\, V^*(s_j) \right]. \tag{10}$$

It is often useful to define the **action-value function** (or Q-function) $Q^*(s_i, a)$, which gives the value of taking action a in state s_i and then acting optimally:

$$Q^*(s_i, a) = r(s_i, a) + \gamma \sum_{j=1}^{n} p_{ij}(a)\, V^*(s_j). \tag{11}$$

The Bellman equation is then $V^*(s_i) = \max_a Q^*(s_i, a)$. The Q-function is the central object in model-free algorithms such as Q-learning (Section 8).

3.5 Solution Methods

The Bellman equation can be solved by several methods. **Value iteration** [2] repeatedly applies the Bellman operator $V_{k+1}(s_i) \leftarrow \max_a [r(s_i, a) + \gamma \sum_j p_{ij}(a)\, V_k(s_j)]$ until convergence; this is the method used in the examples below. **Policy iteration** [28] alternates between evaluating the current policy (solving a linear system) and improving it greedily with respect to the resulting value function; it converges in a finite number of iterations. Both methods yield the optimal policy; value iteration is simpler to implement, while policy iteration often converges in fewer iterations for large state spaces. The MDP can also be formulated as a **linear program** [26], which provides a third exact solution method.

3.6 Examples

3.6.1 Example 1: Basic Binary MDP

Consider an MDP with two states $\mathcal{S} = \{s_1, s_2\}$ and two actions $\mathcal{A} = \{a_1, a_2\}$. The transition probabilities and rewards are:

[3] Bellman's use of b for immediate return should not be confused with the belief state $\mathbf{b}$ introduced in Section 4. We reproduce his original notation here for fidelity to the source.

State	Action	Transitions	Reward
s_1	a_1	$p_{11}(a_1) = 0.8,\ p_{12}(a_1) = 0.2$	$r(s_1, a_1) = 5$
s_1	a_2	$p_{11}(a_2) = 0.3,\ p_{12}(a_2) = 0.7$	$r(s_1, a_2) = 1$
s_2	a_1	$p_{21}(a_1) = 0.6,\ p_{22}(a_1) = 0.4$	$r(s_2, a_1) = 2$
s_2	a_2	$p_{21}(a_2) = 0.1,\ p_{22}(a_2) = 0.9$	$r(s_2, a_2) = 8$

The corresponding decision diagram is shown in Figure 2. Each state node (solid outline) branches into action nodes (dotted outline), which in turn lead probabilistically to successor states.

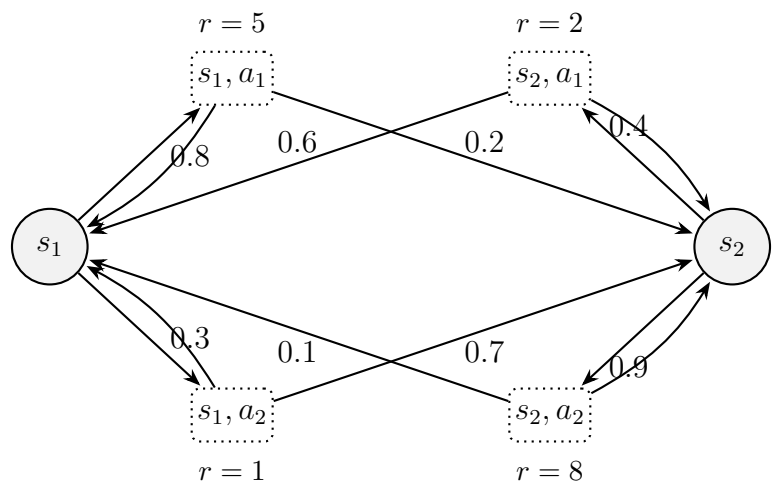

Figure 2: A two-state MDP. Solid circles are states; dotted rectangles are action (decision) nodes labeled (s, a). Edges from action nodes to states are labeled with transition probabilities. Rewards are shown in italics above or below each action node.

With discount factor $\gamma = 0.9$, the Bellman equation gives the system

$$V^*(s_1) = \max\Big\{ \underbrace{5 + 0.9\big[0.8\,V^*(s_1) + 0.2\,V^*(s_2)\big]}_{a_1},\ \underbrace{1 + 0.9\big[0.3\,V^*(s_1) + 0.7\,V^*(s_2)\big]}_{a_2} \Big\}, \tag{12}$$

$$V^*(s_2) = \max\Big\{ \underbrace{2 + 0.9\big[0.6\,V^*(s_1) + 0.4\,V^*(s_2)\big]}_{a_1},\ \underbrace{8 + 0.9\big[0.1\,V^*(s_1) + 0.9\,V^*(s_2)\big]}_{a_2} \Big\}. \tag{13}$$

To solve, we guess that the optimal policy is $\pi^*(s_1) = a_1$ and $\pi^*(s_2) = a_2$ (choosing the high-reward action in each state) and verify. Under this candidate the Bellman equations become linear:

$$V^*(s_1) = 5 + 0.9\big[0.8\,V^*(s_1) + 0.2\,V^*(s_2)\big], \tag{14}$$

$$V^*(s_2) = 8 + 0.9\big[0.1\,V^*(s_1) + 0.9\,V^*(s_2)\big]. \tag{15}$$

Rearranging,

$$(1 - 0.72)\,V^*(s_1) - 0.18\,V^*(s_2) = 5, \tag{16}$$

$$-0.09\,V^*(s_1) + (1 - 0.81)\,V^*(s_2) = 8, \tag{17}$$

i.e.,

$$0.28\,V^*(s_1) - 0.18\,V^*(s_2) = 5, \tag{18}$$

$$-0.09\,V^*(s_1) + 0.19\,V^*(s_2) = 8. \tag{19}$$

Solving yields $V^*(s_1) \approx 64.59$ and $V^*(s_2) \approx 72.70$. To verify optimality, we check that the alternative action does not yield a higher value at either state:

$$\text{At } s_1\text{:} \quad a_2 \implies 1 + 0.9\big[0.3(64.59) + 0.7(72.70)\big] \approx 64.24 < 64.59, \tag{20}$$

$$\text{At } s_2\text{:} \quad a_1 \implies 2 + 0.9\big[0.6(64.59) + 0.4(72.70)\big] \approx 63.05 < 72.70. \tag{21}$$

Since neither alternative improves the value, the candidate policy is confirmed optimal:

$$\pi^*(s_1) = a_1, \qquad \pi^*(s_2) = a_2. \tag{22}$$

The agent should take action a_1 (high immediate reward, likely stay in s_1) when in state s_1, and action a_2 (high immediate reward, likely stay in s_2) when in state s_2.

The following Python code verifies this solution via value iteration:

```
import numpy as np

V = np.array([0.0, 0.0])
gamma = 0.9

for i in range(10000):
    V1_a1 = 5 + gamma * (0.8 * V[0] + 0.2 * V[1])
    V1_a2 = 1 + gamma * (0.3 * V[0] + 0.7 * V[1])
    V2_a1 = 2 + gamma * (0.6 * V[0] + 0.4 * V[1])
    V2_a2 = 8 + gamma * (0.1 * V[0] + 0.9 * V[1])

    V_new = np.array([max(V1_a1, V1_a2), max(V2_a1, V2_a2)])
    if np.max(np.abs(V_new - V)) < 1e-10:
        print(f"Converged at iteration {i}")
        break
    V = V_new

print(f"V*(s1) = {V[0]:.2f}") # 64.59
print(f"V*(s2) = {V[1]:.2f}") # 72.70
```

Listing 1: Value iteration for the two-state MDP example.

3.6.2 Example 2: Simple Asset Maintenance MDP

Consider an asset that is either operational or failed. The agent must decide at each time step whether to do nothing or replace the asset. The MDP is defined by $\mathcal{S} = \{\text{Op}, \text{Fail}\}$, $\mathcal{A} = \{\text{Do Nothing}, \text{Replace}\}$, and discount factor $\gamma = 0.9$. The state value of being operational is $+1$ unit per period and the penalty for being in a failed state is -10 units per period. Replacement costs 5 units regardless of the current state; doing nothing is free. The net reward is therefore $r(s, a) = (\text{state value}) - (\text{action cost})$:

State	Action	Transitions	Reward
Op	Do Nothing	$p(\text{Op}\to\text{Op}) = 0.9,\ p(\text{Op}\to\text{Fail}) = 0.1$	$r = 1$
Op	Replace	$p(\text{Op}\to\text{Op}) = 1.0$	$r = -4$
Fail	Do Nothing	$p(\text{Fail}\to\text{Fail}) = 1.0$	$r = -10$
Fail	Replace	$p(\text{Fail}\to\text{Op}) = 1.0$	$r = -15$

An operational asset that is left alone fails with probability 0.1 each period; once failed, it remains failed until replaced. Replacement always restores the asset to operational.

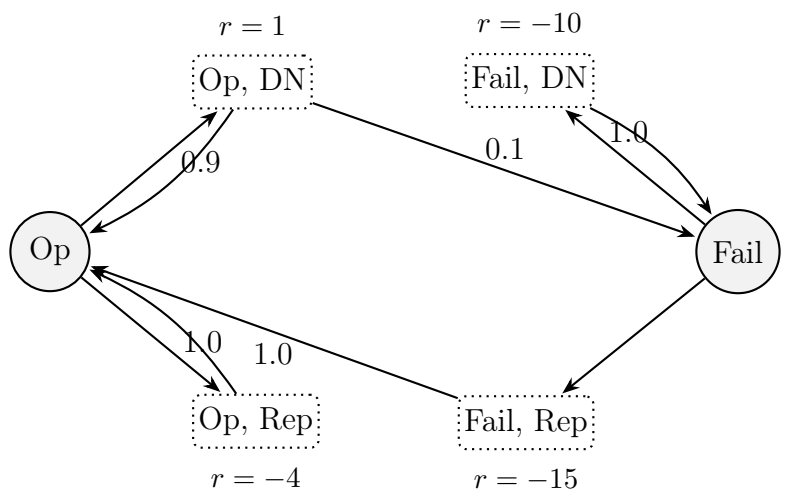

Figure 3: Asset maintenance MDP. DN = Do Nothing, Rep = Replace.

The Bellman equation gives:

$$V^*(\text{Op}) = \max\Big\{ \underbrace{1 + 0.9\big[0.9\,V^*(\text{Op}) + 0.1\,V^*(\text{Fail})\big]}_{\text{Do Nothing}},\ \underbrace{-4 + 0.9\,V^*(\text{Op})}_{\text{Replace}} \Big\}, \tag{23}$$

$$V^*(\text{Fail}) = \max\Big\{ \underbrace{-10 + 0.9\,V^*(\text{Fail})}_{\text{Do Nothing}},\ \underbrace{-15 + 0.9\,V^*(\text{Op})}_{\text{Replace}} \Big\}. \tag{24}$$

The intuition is clear: an operational asset should be left alone (avoiding the unnecessary replacement cost), while a failed asset should be replaced (the one-time cost of 5 is far preferable to accumulating -10 per period indefinitely). We guess $\pi^*(\text{Op}) = \text{Do Nothing}$ and $\pi^*(\text{Fail}) = \text{Replace}$ and verify. Under this candidate:

$$V^*(\text{Op}) = 1 + 0.9\big[0.9\,V^*(\text{Op}) + 0.1\,V^*(\text{Fail})\big], \tag{25}$$

$$V^*(\text{Fail}) = -15 + 0.9\,V^*(\text{Op}). \tag{26}$$

Rearranging,

$$0.19\,V^*(\text{Op}) - 0.09\,V^*(\text{Fail}) = 1, \tag{27}$$

$$-0.9\,V^*(\text{Op}) + V^*(\text{Fail}) = -15. \tag{28}$$

Solving yields $V^*(\text{Op}) \approx -3.21$ and $V^*(\text{Fail}) \approx -17.89$. Both values are negative because maintenance costs and failure penalties erode returns over the infinite horizon. To verify optimality:

$$\text{At Op:} \quad \text{Replace} \implies -4 + 0.9(-3.21) \approx -6.89 < -3.21, \tag{29}$$

$$\text{At Fail:} \quad \text{Do Nothing} \implies -10 + 0.9(-17.89) \approx -26.10 < -17.89. \tag{30}$$

Neither alternative improves the value, confirming the optimal policy:

$$\pi^*(\text{Op}) = \text{Do Nothing}, \qquad \pi^*(\text{Fail}) = \text{Replace}. \tag{31}$$

The following Python code verifies this solution via value iteration:

```
import numpy as np

V = np.array([0.0, 0.0]) # [V(Op), V(Fail)]
gamma = 0.9

for i in range(10000):
    V_op_dn = 1 + gamma * (0.9 * V[0] + 0.1 * V[1])
    V_op_rep = -4 + gamma * (1.0 * V[0])
    V_fl_dn = -10 + gamma * (1.0 * V[1])
    V_fl_rep = -15 + gamma * (1.0 * V[0])

    V_new = np.array([max(V_op_dn, V_op_rep), max(V_fl_dn, V_fl_rep)])
    if np.max(np.abs(V_new - V)) < 1e-10:
        print(f"Converged at iteration {i}")
        break
    V = V_new

print(f"V*(Op) = {V[0]:.2f}") # -3.21
print(f"V*(Fail) = {V[1]:.2f}") # -17.89
```

Listing 2: Value iteration for the asset maintenance MDP.

3.7 Summary

The Markov decision process augments the passive Markov chain with actions and rewards, giving an agent the ability to influence state transitions and accumulate returns. The Bellman equation provides a recursive characterization of the optimal value function, expressing the principle of optimality: the best strategy from any state consists of taking the best immediate action and then continuing optimally. Solving the Bellman equation—via value iteration, policy iteration, or linear programming—yields both the optimal value function and the optimal policy. The two examples illustrate canonical MDP patterns: a binary-choice problem where high immediate reward aligns with favorable transitions, and a maintenance problem where the optimal policy balances operating revenue against replacement cost and failure risk.

3.7.1 Key Equations

Value of state s_i under policy π:

$$V^{\pi}(s_i) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \, r(X_t, \pi(X_t)) \;\middle|\; X_0 = s_i\right].$$

Bellman equation (optimal value function):

$$V^*(s_i) = \max_{a \in \mathcal{A}} \left[r(s_i, a) + \gamma \sum_{j=1}^{n} p_{ij}(a) \, V^*(s_j)\right].$$

Bellman's original finite-horizon recurrence:

$$f_N(s_i) = \max_{a} \left[b(s_i, a) + \sum_{j=1}^{n} a_{ij}(a) \, f_{N-1}(s_j)\right].$$

Optimal policy:

$$\pi^*(s_i) = \arg\max_{a \in \mathcal{A}} \left[r(s_i, a) + \gamma \sum_{j=1}^{n} p_{ij}(a) \, V^*(s_j)\right].$$

Action-value (Q) function:

$$Q^*(s_i, a) = r(s_i, a) + \gamma \sum_{j=1}^{n} p_{ij}(a) \, V^*(s_j).$$

3.7.2 Relevant Applications

- Inventory control and supply chain management, where the decision-maker chooses order quantities based on current stock levels to minimize holding and shortage costs.
- Financial portfolio management, where an investor rebalances assets over time to maximize risk-adjusted returns.
- Autonomous vehicle control, where the vehicle selects acceleration, braking, and steering actions based on its current state to optimize safety and efficiency.
- Game playing (e.g., board games, video games), where an agent selects moves to maximize a score or win probability.
- Energy systems and grid management, where operators decide on generation, storage, and dispatch to balance supply and demand under fluctuating conditions.

4 Partially Observable Markov Decision Processes

In the MDP formulation of Section 3, the agent observes the state directly at each time step. In many real-world problems this assumption is unrealistic: a robot may not know its exact position, a physician cannot observe a patient's internal state, and a maintenance operator may not know whether a machine component has degraded. A **partially observable Markov decision process** (POMDP) extends the MDP by introducing *partial observability*: the agent cannot see the true state but instead receives noisy observations that depend probabilistically on it. This formulation was introduced by Åström [3] and developed algorithmically by Smallwood and Sondik [4].

4.1 Notation

Symbol	Description
$\Omega = \{o_1, \ldots, o_k\}$	Finite set of observations
$O(o \mid s', a)$	Observation probability: $P(\text{observe } o \mid \text{arrive in } s', \text{take } a)$
$\mathbf{b}$	Belief state, a distribution over $\mathcal{S}$
$b(s_i)$	Probability assigned to state s_i in belief $\mathbf{b}$
$b'(s_j)$	Updated belief after action a and observation o
$\rho(\mathbf{b}, a)$	Expected immediate reward under belief $\mathbf{b}$ and action a
$V^*(\mathbf{b})$	Optimal value function over the belief simplex
$P(o \mid \mathbf{b}, a)$	Probability of observation o given belief $\mathbf{b}$ and action a
$\mathbf{b}'_{a,o}$	Updated belief after taking a and observing o

4.2 Formal Definition

A POMDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, p, r, \Omega, O, \gamma, \mathbf{b}_0)$, where $\mathbf{b}_0$ is the initial belief distribution over $\mathcal{S}$:

- $\mathcal{S}$, $\mathcal{A}$, $p_{ij}(a)$, $r(s_i, a)$, and γ: as in the MDP of Section 3.
- $\Omega = \{o_1, o_2, \ldots, o_k\}$: a finite set of **observations** the agent may receive.
- $O(o \mid s', a)$: the **observation function**, giving the probability of receiving observation o after taking action a and arriving in state s'. For each s' and a,

$$O(o \mid s', a) \geq 0 \quad \text{and} \quad \sum_{o \in \Omega} O(o \mid s', a) = 1. \tag{32}$$

When $|\Omega| = |\mathcal{S}|$ and $O(o_i \mid s_i, a) = 1$ for all a, each observation reveals the state exactly and the POMDP reduces to an MDP.

4.3 Belief States

Since the agent cannot observe the state directly, it maintains a **belief state b**, a probability distribution over $\mathcal{S}$:

$$\mathbf{b} = (b(s_1), b(s_2), \ldots, b(s_n)), \quad b(s_i) \geq 0, \quad \sum_{i=1}^{n} b(s_i) = 1, \tag{33}$$

where $b(s_i)$ is the agent's probability that the current state is s_i. After taking action a and receiving observation o, the belief is updated via Bayes' rule:

$$b'(s_j) = \frac{O(o \mid s_j, a) \sum_{i=1}^{n} p_{ij}(a)\, b(s_i)}{\sum_{l=1}^{n} O(o \mid s_l, a) \sum_{i=1}^{n} p_{il}(a)\, b(s_i)}. \tag{34}$$

The numerator first propagates the belief forward through the transition model (prediction), then weights by the observation likelihood (update). The denominator normalizes.

The belief state is a *sufficient statistic*: it captures all information from the history of actions and observations that is relevant to future decisions.

4.4 The Value Function and Bellman's Equation

The key insight, due to Åström [3], is that a POMDP can be reformulated as a fully observable MDP over the *belief space*—the $(n-1)$-dimensional probability simplex. In this **belief MDP**, the "state" is the belief $\mathbf{b}$, the expected immediate reward for taking action a is

$$\rho(\mathbf{b}, a) = \sum_{s \in \mathcal{S}} b(s)\, r(s, a), \tag{35}$$

and the Bellman equation becomes

$$V^*(\mathbf{b}) = \max_{a \in \mathcal{A}} \left[\rho(\mathbf{b}, a) + \gamma \sum_{o \in \Omega} P(o \mid \mathbf{b}, a)\, V^*(\mathbf{b}'_{a,o}) \right], \tag{36}$$

where $P(o \mid \mathbf{b}, a) = \sum_j O(o \mid s_j, a) \sum_i p_{ij}(a)\, b(s_i)$ is the probability of observing o, and $\mathbf{b}'_{a,o}$ is the updated belief from (34).

Unlike the MDP Bellman equation, which ranges over a finite set of states, this equation ranges over a *continuous* belief space. For finite-horizon problems, Smallwood and Sondik [4] showed that V^* is piecewise linear and convex over the belief simplex, which enables exact solution via alpha-vector methods. In general, however, exact finite-horizon POMDP planning is PSPACE-hard [5], and practical algorithms rely on point-based approximations or online search.

4.5 Example: The Tiger Problem

The **Tiger Problem** [6] is a canonical POMDP example. An agent faces two closed doors. Behind one door is a tiger (penalty -100); behind the other is a treasure (reward $+10$). The agent does not know which door hides the tiger but may *listen* for a small cost (-1) to receive a noisy clue.

Formally: $\mathcal{S} = \{\text{TL}, \text{TR}\}$ (tiger left / tiger right), $\mathcal{A} = \{\text{listen}, \text{open-left}, \text{open-right}\}$, and $\Omega = \{\text{HL}, \text{HR}\}$ (hear left / hear right). Listening is 85% accurate:

$$O(\text{HL} \mid \text{TL}, \text{listen}) = 0.85, \quad O(\text{HR} \mid \text{TL}, \text{listen}) = 0.15, \tag{37}$$

and symmetrically for state TR. The listen action leaves the tiger's position unchanged (identity transition: $p_{\text{TL},\text{TL}}(\text{listen}) = p_{\text{TR},\text{TR}}(\text{listen}) = 1$). Opening a door yields $+10$ (treasure) or -100 (tiger) and resets the problem with the tiger placed uniformly at random; observations for open actions are deterministic, revealing whether the tiger or treasure was found.

Rewards:

Action	Tiger behind chosen door	Tiger behind other door
Listen	-1	-1
Open door	-100	$+10$

The structure of the Tiger Problem is shown in Figure 4.

Starting from the uniform belief $b(\text{TL}) = 0.5$, the agent listens and hears the tiger on the left. Applying (34):

$$b'(\text{TL}) = \frac{0.85 \times 0.5}{0.85 \times 0.5 + 0.15 \times 0.5} = 0.85. \tag{38}$$

After a second listen that again indicates left:

$$b''(\text{TL}) = \frac{0.85 \times 0.85}{0.85 \times 0.85 + 0.15 \times 0.15} \approx 0.97. \tag{39}$$

Now sufficiently confident the tiger is on the left, the agent opens the right door.

The optimal policy has an intuitive structure: when the belief is near 0.5 (high uncertainty), listen to gather information; once the belief crosses a threshold (high confidence), open the door away from the tiger. This illustrates a central theme of POMDPs—*information has value*. The listen action has negative immediate reward but reduces uncertainty, enabling better future decisions. This tradeoff between gathering information and acting on it is the hallmark of decision-making under partial observability.

4.6 Summary

The POMDP extends the MDP to settings where the agent cannot observe the true state and must instead rely on noisy observations. The key insight is that the agent's belief—a

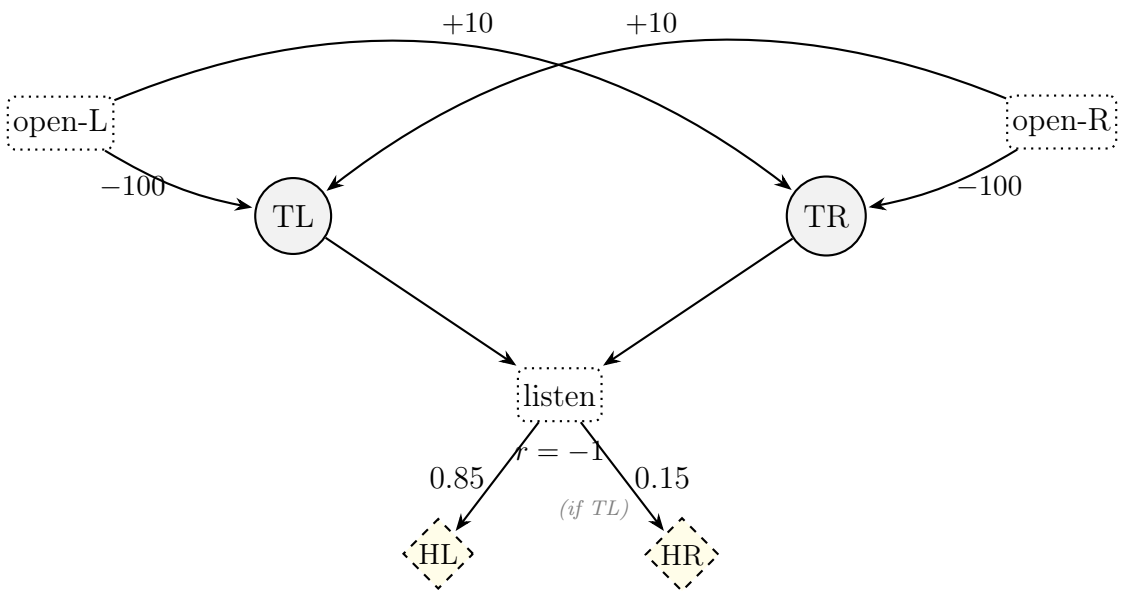

Figure 4: The Tiger Problem. States TL (tiger left) and TR (tiger right) are hidden. The listen action yields a noisy observation (dashed diamonds): hearing HL with probability 0.85 when the tiger is left. Opening a door yields +10 (treasure) or −100 (tiger) and resets the problem.

probability distribution over states, updated via Bayes' rule—serves as a sufficient statistic, transforming the POMDP into a continuous-state MDP over the belief simplex. This belief MDP preserves the Bellman equation structure but over an uncountably infinite state space, making exact finite-horizon planning PSPACE-complete in general. The Tiger Problem illustrates the core tension: information-gathering actions (listening) have negative immediate reward but reduce uncertainty, enabling higher-value decisions later. This *value of information* is the defining concept of partially observable decision-making.

4.6.1 Key Equations

Belief state update (Bayes' rule):

$$b'(s_j) = \frac{O(o \mid s_j, a) \sum_{i=1}^{n} p_{ij}(a)\, b(s_i)}{\sum_{l=1}^{n} O(o \mid s_l, a) \sum_{i=1}^{n} p_{il}(a)\, b(s_i)}.$$

Expected immediate reward under belief $\mathbf{b}$:

$$\rho(\mathbf{b}, a) = \sum_{s \in \mathcal{S}} b(s)\, r(s, a).$$

Bellman equation over the belief simplex:

$$V^*(\mathbf{b}) = \max_{a \in \mathcal{A}} \left[\rho(\mathbf{b}, a) + \gamma \sum_{o \in \Omega} P(o \mid \mathbf{b}, a)\, V^*(\mathbf{b}'_{a,o}) \right].$$

4.6.2 Relevant Applications

- Robot navigation and localization, where the robot infers its position from noisy sensor readings (e.g., lidar, GPS) while planning a path.
- Medical diagnosis and treatment planning, where the physician cannot directly observe the patient's disease state and must order tests (observations) before deciding on treatment.
- Spoken dialogue systems, where the system maintains a belief over the user's intent based on noisy speech recognition output.
- Aircraft collision avoidance, where the positions and intents of nearby aircraft are uncertain and must be inferred from radar observations.
- Inventory management under demand uncertainty, where the true demand distribution is unobserved and must be estimated from sales data.

5 Multi-Armed Bandit Problems

The **multi-armed bandit** (MAB) problem isolates a fundamental challenge that appears throughout the preceding sections: the tradeoff between *exploration* (gathering information) and *exploitation* (acting on current knowledge). The name comes from slot machines, colloquially called "one-armed bandits." An agent faces K such machines, each with an unknown reward distribution, and must decide which arm to pull at each time step to maximize cumulative reward.

In the frequentist formulation, the MAB can be viewed as a degenerate MDP with a single observable state: the only decision is which action (arm) to take, and there are no state transitions. In the Bayesian formulation, however, the agent's posterior beliefs over arm parameters evolve with each observation, making the problem an MDP over belief states—and it is this richer formulation that gives rise to the Gittins index. The MAB can also be viewed as a POMDP in which the hidden state is the vector of true arm means and each reward observation provides partial information about them; the Bayesian perspective connects directly to the belief-state framework of Section 4.

The problem was first studied by Thompson [7] and formalized in its modern form by Robbins [8]. Lattimore and Szepesvári [29] provide a comprehensive modern treatment.

5.1 Notation

Symbol	Description
K	Number of arms
μ_i	True mean reward of arm i
T	Time horizon (number of rounds)
I_t	Arm selected at round t
$X_{I_t,t}$	Reward received at round t
R_T	Cumulative regret over T rounds
μ^*	Mean of the best arm, $\max_i \mu_i$
Δ_i	Gap (suboptimality) of arm i, $\mu^* - \mu_i$
$T_i(T)$	Number of times arm i is pulled in T rounds
ε	Exploration probability (ε-greedy)
$\hat{\mu}_i$	Empirical mean reward of arm i
N_i	Number of pulls of arm i
θ_i	True success probability of Bernoulli arm i
S_i	Number of observed successes on arm i
F_i	Number of observed failures on arm i
$\tilde{\theta}_i$	Posterior sample for arm i (Thompson Sampling)
G_i	Gittins index of arm i
$G(\alpha, \beta, \gamma)$	Gittins index for a Beta(α, β) arm with discount γ; α, β here are Beta shape parameters, not as in Section 2
$\mathrm{KL}(\nu_i, \nu^*)$	Kullback–Leibler divergence, arm i's distribution to optimal

5.2 Formal Definition

An agent faces K arms. Each arm $i \in \{1, \ldots, K\}$ generates i.i.d. rewards from an unknown distribution with mean μ_i. At each round $t = 1, 2, \ldots, T$, the agent selects an arm I_t and receives reward $X_{I_t,t}$. The agent's goal is to maximize cumulative reward, or equivalently to minimize **cumulative regret**:

$$R_T = T\mu^* - \sum_{t=1}^{T} \mathbb{E}[X_{I_t,t}] = \sum_{i:\, \mu_i < \mu^*} \Delta_i \cdot \mathbb{E}[T_i(T)], \tag{40}$$

where $\mu^* = \max_i \mu_i$ is the mean of the best arm, $\Delta_i = \mu^* - \mu_i$ is the **gap** (suboptimality) of arm i, and $T_i(T)$ is the number of times arm i is pulled in T rounds. Regret measures how much reward is lost by not always pulling the best arm.

Remark (frequentist vs. Bayesian regret). The definition above is *frequentist* (or worst-case) regret: the arm means $\mu_1, \ldots, \mu_K$ are fixed unknowns, and the expectation is over the algorithm's randomness. In the Bayesian formulation, the arm means are drawn from a prior distribution and *Bayesian regret* averages over both the prior and the algorithm: $\mathbb{E}_{\boldsymbol{\mu}}[\mathbb{E}[R_T \mid \boldsymbol{\mu}]]$. The Lai–Robbins lower bound and the UCB1 guarantee stated below are

frequentist results; the Gittins index is optimal in the Bayesian sense; Thompson Sampling achieves strong guarantees in both frameworks.

Figure 5 illustrates the structure.

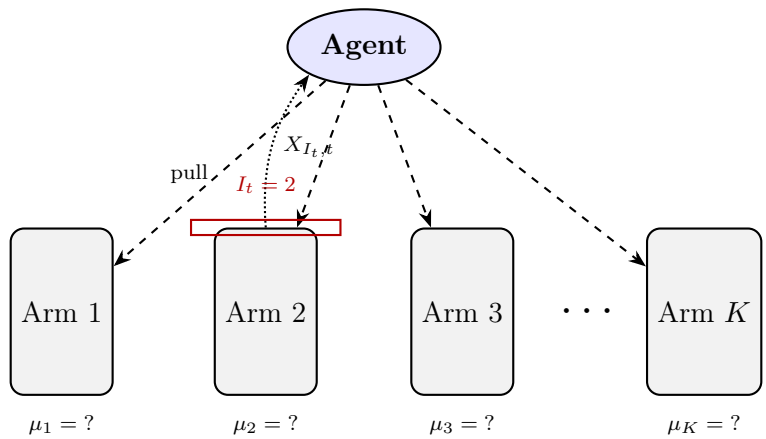

Figure 5: The multi-armed bandit. An agent selects one of K arms at each round. The chosen arm (highlighted) returns a stochastic reward $X_{I_t,t}$; the true means μ_i are unknown and must be learned through interaction.

5.3 Strategies

5.3.1 ε-Greedy

The simplest bandit strategy is ε-greedy: with probability $1-\varepsilon$, pull the arm with the highest empirical mean (exploit); with probability ε, pull an arm uniformly at random (explore). While intuitive, a fixed ε yields linear regret because the agent continues exploring at rate ε indefinitely, even after the best arm is identified. Decaying $\varepsilon_t \propto t^{-1/3}$ improves this to $O(T^{2/3})$ [11], but this is still far worse than the $O(\ln T)$ achievable by the methods below.

5.3.2 Upper Confidence Bound (UCB1)

The UCB1 algorithm [11] embodies the principle of *optimism in the face of uncertainty*. After pulling each arm once, at every subsequent round the agent selects

$$I_t = \arg\max_{i\in\{1,\dots,K\}} \left[\hat{\mu}_i + \sqrt{\frac{2\ln t}{N_i}}\right], \tag{41}$$

where $\hat{\mu}_i$ is the empirical mean reward of arm i and N_i is the number of times it has been pulled. The first term exploits current estimates; the second is an *exploration bonus* that inflates the index of under-sampled arms. Arms with fewer pulls have wider confidence intervals and thus receive higher indices, ensuring they are eventually tried.

5.3.3 Thompson Sampling

Thompson Sampling [7] takes a Bayesian approach. The agent maintains a posterior distribution over each arm's mean and, at each round, *samples* from these posteriors and pulls the arm whose sample is highest.

For Bernoulli arms with unknown success probability θ_i, a natural prior is $\theta_i \sim \text{Beta}(1, 1)$ (uniform). After observing S_i successes and F_i failures on arm i, the posterior is $\theta_i \sim \text{Beta}(S_i + 1, F_i + 1)$. The algorithm is:

1. For each arm i, sample $\tilde{\theta}_i \sim \text{Beta}(S_i + 1, F_i + 1)$.
2. Pull arm $I_t = \arg\max_i \tilde{\theta}_i$.
3. Observe reward $r_t \in \{0, 1\}$ and update: if $r_t = 1$, set $S_{I_t} \leftarrow S_{I_t} + 1$; otherwise $F_{I_t} \leftarrow F_{I_t} + 1$.

When the posterior is concentrated on a high mean, the samples are likely to be high (exploitation); when it is diffuse, samples are variable and the arm may be tried (exploration). The Beta–Bernoulli conjugacy ensures the posterior update is a simple counter increment.

5.3.4 The Gittins Index

For the discounted infinite-horizon Bayesian bandit with independent, rested arms (discount factor γ, known prior on each arm), Gittins [9] proved a remarkable result: the optimal policy is an *index policy*. Each arm i is assigned a **Gittins index** G_i computed from its posterior alone, and the agent always pulls the arm with the highest index. The index can be interpreted as the fair price at which the agent would be indifferent between continuing to pull the arm and receiving a guaranteed constant payoff forever. The Gittins index theorem shows that the multi-arm problem decomposes into independent single-arm problems—a dramatic simplification. This optimality holds specifically for the discounted infinite-horizon objective with independent, rested arms and a known prior; it breaks down under finite horizons, average-reward criteria, or when arms evolve even while not being pulled—motivating the restless bandit model of Section 7.

5.3.5 Successive Elimination

Rather than selecting arms via an index, **successive elimination** [14] maintains a set of *active* arms and permanently discards any arm whose upper confidence bound falls below another arm's lower confidence bound. Once eliminated, an arm is never pulled again. The algorithm achieves $O(K \ln T)$ regret while offering a conceptually different mechanism: instead of balancing exploration and exploitation continuously, it reduces the problem size over time.

5.3.6 KL-UCB

KL-UCB [15] replaces the Hoeffding-based confidence bound in UCB1 with a tighter bound derived from the Kullback–Leibler divergence. At each round the agent selects $I_t = \arg\max_i \sup\{q : N_i \cdot \mathrm{KL}(\hat{\mu}_i, q) \leq \ln t\}$, where $\mathrm{KL}(\hat{\mu}_i, q)$ is the KL divergence between Bernoulli distributions with parameters $\hat{\mu}_i$ and q. The resulting bound is asymptotically optimal, matching the Lai–Robbins lower bound with the exact constant—an improvement over UCB1, which matches only up to a multiplicative factor.

5.3.7 EXP3

All strategies above assume *stochastic* rewards (i.i.d. draws from fixed distributions). **EXP3** (Exponential-weight algorithm for Exploration and Exploitation) [13] handles the *adversarial* setting, where rewards may be chosen by an adversary with no distributional assumption. The algorithm maintains a probability distribution over arms, updated via exponential weights on estimated rewards, and achieves $O(\sqrt{KT \ln K})$ regret against the best fixed arm in hindsight.

5.4 Regret Bounds

Lai and Robbins [10] established a fundamental **lower bound** on regret. For any consistent policy[4],

$$\liminf_{T\to\infty} \frac{\mathbb{E}[R_T]}{\ln T} \geq \sum_{i:\, \mu_i < \mu^*} \frac{\Delta_i}{\mathrm{KL}(\nu_i, \nu^*)}, \tag{42}$$

where $\mathrm{KL}(\nu_i, \nu^*)$ is the Kullback–Leibler divergence from arm i's distribution to the optimal arm's distribution. This says regret must grow at least logarithmically in T, and distinguishing a suboptimal arm from the best requires more pulls when their distributions are similar (small KL divergence).

UCB1 achieves a finite-time upper bound of

$$\mathbb{E}[R_T] \leq \sum_{i:\, \mu_i < \mu^*} \frac{8 \ln T}{\Delta_i} + \left(1 + \frac{\pi^2}{3}\right) \sum_i \Delta_i, \tag{43}$$

which is $O(K \ln T)$—logarithmic in T and matching the Lai–Robbins lower bound up to constant factors for bounded rewards. Thompson Sampling also achieves asymptotically optimal regret under standard regularity conditions on the prior [12], with expected regret

$$\mathbb{E}[R_T] = O\left(\sum_{i:\, \mu_i < \mu^*} \frac{\Delta_i \ln T}{\mathrm{KL}(\nu_i, \nu^*)}\right), \tag{44}$$

[4] A policy is *consistent* if $\mathbb{E}[T_i(T)] = o(T^\alpha)$ for every suboptimal arm i and every $\alpha > 0$.

matching the Lai–Robbins lower bound up to constant factors. Figure 6 illustrates the qualitative separation between these growth rates.

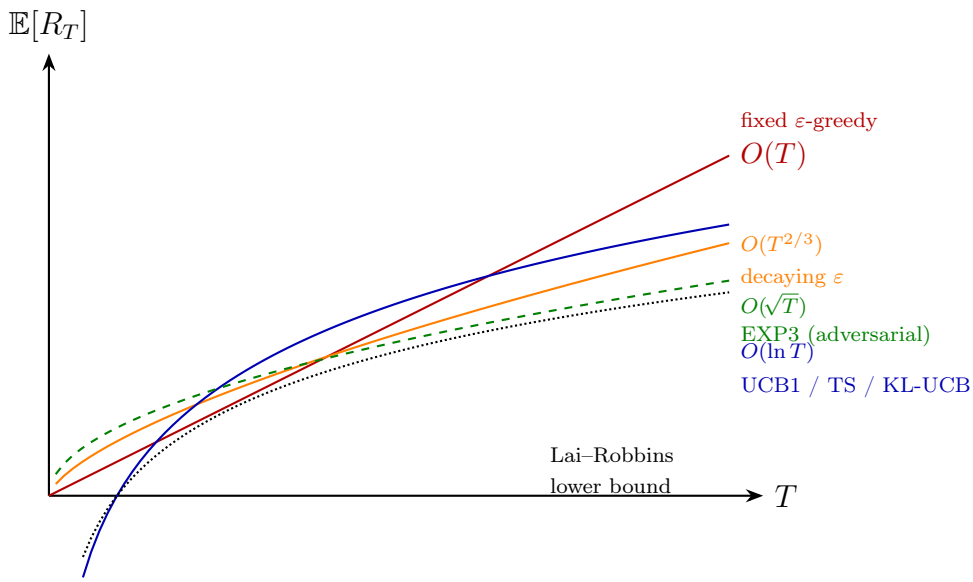

Figure 6: Qualitative regret growth rates for bandit strategies (not to scale). The Lai–Robbins lower bound (dotted) shows that $O(\ln T)$ is the best achievable rate for stochastic bandits. Fixed ε-greedy incurs linear regret; decaying ε achieves $O(T^{2/3})$; UCB1, Thompson Sampling, and KL-UCB achieve the optimal $O(\ln T)$ rate. EXP3 achieves $O(\sqrt{T})$ in the harder adversarial setting.

5.5 Examples

We illustrate each strategy on a common instance: $K = 3$ arms with Bernoulli success probabilities $\theta_1 = 0.3$, $\theta_2 = 0.7$, and $\theta_3 = 0.5$. Arm 2 is optimal; the agent does not know the θ_i and must learn from rewards.

5.5.1 ε-Greedy

Set $\varepsilon = 0.3$. After initializing by pulling each arm once, the agent exploits (pulls the arm with the highest empirical mean) with probability 0.7 and explores (pulls a uniformly random arm) with probability 0.3.

t	Arm	Type	Reward	$\hat{\mu} = (\hat{\mu}_1, \hat{\mu}_2, \hat{\mu}_3)$
1	1	init	0	(0.00, −, −)
2	2	init	1	(0.00, 1.00, −)
3	3	init	1	(0.00, 1.00, 1.00)
4	2	exploit	1	(0.00, 1.00, 1.00)
5	2	exploit	0	(0.00, 0.67, 1.00)
6	3	exploit	0	(0.00, 0.67, 0.50)
7	2	exploit	1	(0.00, 0.75, 0.50)
8	1	**explore**	0	(0.00, 0.75, 0.50)
9	2	exploit	1	(0.00, 0.80, 0.50)
10	3	**explore**	0	(0.00, 0.80, 0.33)

After ten rounds the pull counts are $N = (2, 5, 3)$. Arm 2 has been pulled most often, and its empirical mean (0.80) is close to the true $\theta_2 = 0.7$. But notice the two explore rounds ($t = 8, 10$): at $t = 8$, the agent wastes a pull on arm 1 despite strong evidence it is the worst arm. This is the fundamental limitation of ε-greedy—it explores *uniformly at random*, directing equal attention to clearly bad and merely uncertain arms.

5.5.2 UCB1

We trace UCB1 through seven rounds on the same instance.

Rounds 1–3 (initialization): Pull each arm once.

t	Arm	Reward	$\hat{\mu}_i$
1	1	1	(1.00, −, −)
2	2	1	(1.00, 1.00, −)
3	3	0	(1.00, 1.00, 0.00)

Rounds 4–7 (UCB1 selection): At each round, compute the index $\hat{\mu}_i + \sqrt{2 \ln t / N_i}$ and pull the arm with the highest value.

t	UCB_1	UCB_2	UCB_3	Arm	Reward
4	**2.67**	2.67	1.67	1	0
5	1.77	**2.79**	1.79	2	1
6	1.84	**2.34**	1.89	2	0
7	1.90	1.81	**1.97**	3	1

After seven rounds the pull counts are $N = (2, 3, 2)$ and the empirical means are $\hat{\mu} = (0.50, 0.67, 0.50)$. Already arm 2 (the true best) has been pulled most often, while the exploration bonus ensures that arm 3 is revisited at $t = 7$ when its uncertainty grows large enough. Unlike ε-greedy, UCB1 directs exploration toward arms whose *uncertainty* is high, not at random.

5.5.3 Thompson Sampling

Each arm begins with a uniform prior $\theta_i \sim \text{Beta}(1,1)$. At each round, the agent samples from the current posteriors and pulls the arm with the highest sample.

t	$\tilde{\theta}_1$	$\tilde{\theta}_2$	$\tilde{\theta}_3$	Arm	Reward	Posteriors
1	0.42	**0.88**	0.57	2	1	$\text{B}(1,1),\ \text{B}(2,1),\ \text{B}(1,1)$
2	0.71	0.60	**0.82**	3	0	$\text{B}(1,1),\ \text{B}(2,1),\ \text{B}(1,2)$
3	0.65	**0.94**	0.28	2	1	$\text{B}(1,1),\ \text{B}(3,1),\ \text{B}(1,2)$
4	**0.85**	0.78	0.14	1	0	$\text{B}(1,2),\ \text{B}(3,1),\ \text{B}(1,2)$
5	0.18	**0.72**	0.53	2	1	$\text{B}(1,2),\ \text{B}(4,1),\ \text{B}(1,2)$
6	0.33	**0.91**	0.27	2	0	$\text{B}(1,2),\ \text{B}(4,2),\ \text{B}(1,2)$
7	0.09	**0.67**	0.42	2	1	$\text{B}(1,2),\ \text{B}(5,2),\ \text{B}(1,2)$

After seven rounds the pull counts are $N = (1,5,1)$. The posterior for arm 2 is $\text{Beta}(5,2)$, with mean $5/7 \approx 0.71$—close to the true $\theta_2 = 0.7$—while the posteriors for arms 1 and 3 remain diffuse. At $t = 4$, arm 1's diffuse posterior $\text{Beta}(1,1)$ happens to produce a high sample (0.85), triggering an exploration pull; the resulting failure shifts the posterior to $\text{Beta}(1,2)$, making future high samples less likely. This illustrates how Thompson Sampling *self-corrects*: it explores in proportion to the posterior probability that each arm is optimal, naturally reducing exploration of inferior arms as evidence accumulates.

5.5.4 The Gittins Index

The Gittins index is defined for the discounted Bayesian setting and requires computing the value of optimally playing a single arm against a known alternative—a calculation that does not reduce to a simple round-by-round trace. For a Beta–Bernoulli arm with posterior $\text{Beta}(\alpha,\beta)$ (where α,β are the shape parameters of the Beta distribution, not the transition probabilities of Section 2) and discount factor γ, the index $G(\alpha,\beta,\gamma)$ can be computed numerically via dynamic programming or calibration. Tables of Gittins indices show, for example, that $G(1,1,0.9) \approx 0.7025$, meaning an arm with a uniform prior is worth playing as long as its "retirement reward" is below roughly 0.70. As successes accumulate the index rises (e.g., $G(5,2,0.9) \approx 0.7867$); as failures accumulate it falls (e.g., $G(1,2,0.9) \approx 0.5000$). The agent always plays the arm with the highest current index, achieving the Bayes-optimal policy without ever computing a joint posterior over all arms.

5.6 Summary

The multi-armed bandit distills the exploration–exploitation tradeoff into its purest form: a single-state decision problem where the agent must learn which action is best by trying

them. The ε-greedy strategy offers simplicity but wastes exploration on clearly inferior arms; UCB1 directs exploration toward uncertain arms using confidence bounds; Thompson Sampling leverages Bayesian posteriors to explore in proportion to each arm's probability of being optimal; and the Gittins index provides the Bayes-optimal solution for the discounted infinite-horizon case by reducing the multi-arm problem to independent single-arm computations. The Lai–Robbins lower bound establishes that logarithmic regret growth is the best any consistent policy can achieve, and both UCB1 and Thompson Sampling attain this rate. Extensions such as KL-UCB sharpen the constants, successive elimination reduces the active arm set over time, and EXP3 handles adversarial (non-stochastic) environments.

5.6.1 Key Equations

Cumulative regret:

$$R_T = T\mu^* - \sum_{t=1}^{T} \mathbb{E}[X_{I_t,t}] = \sum_{i:\,\mu_i<\mu^*} \Delta_i \cdot \mathbb{E}[T_i(T)].$$

UCB1 arm selection:

$$I_t = \arg\max_{i\in\{1,\dots,K\}} \left[\hat{\mu}_i + \sqrt{\frac{2\ln t}{N_i}}\right].$$

Lai–Robbins lower bound:

$$\liminf_{T\to\infty} \frac{\mathbb{E}[R_T]}{\ln T} \geq \sum_{i:\,\mu_i<\mu^*} \frac{\Delta_i}{\mathrm{KL}(\nu_i,\nu^*)}.$$

UCB1 upper bound:

$$\mathbb{E}[R_T] \leq \sum_{i:\,\mu_i<\mu^*} \frac{8\ln T}{\Delta_i} + \left(1+\frac{\pi^2}{3}\right)\sum_i \Delta_i.$$

Thompson Sampling posterior update (Bernoulli arms):

$$\theta_i \mid \text{data} \sim \mathrm{Beta}(S_i+1,\ F_i+1).$$

5.6.2 Relevant Applications

- Clinical trial design, where treatments (arms) must be allocated to patients to identify the most effective therapy while minimizing harm.
- Online advertising and recommendation systems, where the platform must balance showing proven content against testing new alternatives.

- A/B testing and website optimization, where variants of a webpage or feature are compared in real time.
- Dynamic pricing, where a seller experiments with price points to learn demand curves while maximizing revenue.
- Network routing and server selection, where traffic must be allocated across paths or servers with unknown performance characteristics.

6 Contextual Bandit Problems

The **contextual bandit** problem extends the multi-armed bandit framework of Section 5 by incorporating *side information*—a context vector observed before each arm selection. The context may encode user features in a recommender system, patient characteristics in personalized medicine, or the current state of a bidding environment in online advertising. Unlike a full MDP, the context arrives exogenously at each round and does not depend on past actions; the agent observes a context, selects an arm, and receives a reward, but there are no state transitions. Contextual bandits thus occupy a middle ground between the stateless exploration of Section 5 and the full sequential planning of Section 3.

The contextual bandit model was formalized by Langford and Zhang [20], who introduced the Epoch-Greedy algorithm, and has since become a cornerstone of online learning in applications ranging from news article recommendation to clinical trial design. A particularly influential algorithm is LinUCB [21], which applies the principle of optimism in the face of uncertainty to a linear reward model.

The key algorithmic challenge is the same as in standard bandits—balancing exploration and exploitation—but now the agent must generalize across contexts. A policy that performs well in one context may be suboptimal in another, and the agent must learn a mapping from contexts to arm selections.

6.1 Notation

Symbol	Description
$\mathcal{X} \subseteq \mathbb{R}^d$	Context space
x_t	Context vector observed at round t
K	Number of arms (actions)
$\boldsymbol{\phi}(x, a)$	Feature vector for context–arm pair (x, a)
d	Dimension of feature vectors
$\boldsymbol{\theta}^*$	Unknown parameter vector
$r_t(a)$	Reward for arm a at round t
η_t	Zero-mean noise in reward model
$\pi : \mathcal{X} \to \mathcal{A}$	Policy mapping contexts to arms
R_T	Cumulative regret over T rounds
$\mathbf{V}_t$	Design matrix (covariance estimate) at round t
$\mathbf{b}_t$	Cumulative reward-weighted features at round t[5]
$\hat{\boldsymbol{\theta}}_t$	Least-squares estimate of $\boldsymbol{\theta}^*$ at round t
α	Exploration parameter (confidence width)[6]
λ	Regularization parameter[7]
σ^2	Noise variance (Thompson Sampling)
σ_θ^2	Prior variance for $\boldsymbol{\theta}$
$\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t$	Posterior mean and covariance of $\boldsymbol{\theta}$[8]
$\tilde{\boldsymbol{\theta}}$	Posterior sample (Thompson Sampling)
N	Number of experts (EXP4)
$w_{i,t}$	Weight of expert i at round t (EXP4)
$p_t(a)$	Mixing probability over arms at round t (EXP4)
$\hat{r}_t(a)$	Importance-weighted reward estimate (EXP4)
η	Learning rate (EXP4)[9]

6.2 Formal Definition

The contextual bandit proceeds in rounds $t = 1, 2, \ldots, T$. At each round:

1. Nature reveals a context $x_t \in \mathcal{X}$.
2. The agent selects an arm $a_t \in \{1, \ldots, K\}$, possibly based on the context and past observations.
3. The agent receives reward $r_t(a_t)$ and observes no reward for the other arms.

[6] Not to be confused with the belief state $\mathbf{b}$ of Section 4 or the immediate return b in Bellman's notation (Section 3).

[7] This α is unrelated to the transition probability α in Section 2 or the Beta shape parameters in Section 5.

[8] This λ is unrelated to the Lagrange multiplier λ in Section 7.

[9] Not to be confused with the state distribution $\boldsymbol{\mu}_t$ in Section 2.

[10] The EXP4 learning rate η is unrelated to the noise variable η_t in the linear reward model above.

In the **stochastic linear model**, the expected reward for arm a given context x is

$$\mathbb{E}[r_t(a) \mid x_t] = \boldsymbol{\phi}(x_t, a)^\top \boldsymbol{\theta}^*, \tag{45}$$

where $\boldsymbol{\phi} : \mathcal{X} \times \{1, \ldots, K\} \to \mathbb{R}^d$ is a known feature map and $\boldsymbol{\theta}^* \in \mathbb{R}^d$ is an unknown parameter vector. The actual reward is often modeled as

$$r_t(a) = \boldsymbol{\phi}(x_t, a)^\top \boldsymbol{\theta}^* + \eta_t, \tag{46}$$

where η_t is zero-mean noise (e.g., sub-Gaussian).

The agent's goal is to minimize **cumulative regret**, defined as

$$R_T = \sum_{t=1}^{T} \left[\max_{a \in \{1,\ldots,K\}} \boldsymbol{\phi}(x_t, a)^\top \boldsymbol{\theta}^* - \boldsymbol{\phi}(x_t, a_t)^\top \boldsymbol{\theta}^* \right], \tag{47}$$

the total difference between the reward of the optimal arm in each context and the arm actually chosen. Unlike the standard bandit setting, the optimal arm $a^*(x_t) = \arg\max_a \boldsymbol{\phi}(x_t, a)^\top \boldsymbol{\theta}^*$ may change from round to round depending on the context.

Two common settings for the context arrival process are:

- **Stochastic contexts**: $x_1, x_2, \ldots$ are drawn i.i.d. from some fixed distribution over $\mathcal{X}$.
- **Adversarial contexts**: $x_1, x_2, \ldots$ are chosen by an adversary who may observe the agent's past behavior but not its future randomization.

6.3 LinUCB: Linear Upper Confidence Bound

LinUCB [21] is the most widely-used contextual bandit algorithm in practice. It extends the optimistic principle behind UCB1 to linear reward models, maintaining uncertainty estimates via ridge regression. Where UCB1 estimates a fixed unknown mean for each arm independently, LinUCB models each arm's reward as a linear function of the observed context, enabling generalization across similar contexts.

At each round, LinUCB maintains a ridge-regression estimate of $\boldsymbol{\theta}^*$. Let $\mathbf{V}_t \in \mathbb{R}^{d \times d}$ denote the design matrix and $\mathbf{b}_t \in \mathbb{R}^d$ the cumulative reward-weighted features after t rounds:

$$\mathbf{V}_t = \lambda \mathbf{I} + \sum_{\tau=1}^{t-1} \boldsymbol{\phi}(x_\tau, a_\tau) \boldsymbol{\phi}(x_\tau, a_\tau)^\top, \quad \mathbf{b}_t = \sum_{\tau=1}^{t-1} r_\tau \boldsymbol{\phi}(x_\tau, a_\tau), \tag{48}$$

where $\lambda > 0$ is a regularization parameter. The ridge estimate is $\hat{\boldsymbol{\theta}}_t = \mathbf{V}_t^{-1} \mathbf{b}_t$.

The upper confidence bound for arm a at round t is

$$\mathrm{UCB}_t(a) = \boldsymbol{\phi}(x_t, a)^\top \hat{\boldsymbol{\theta}}_t + \alpha \sqrt{\boldsymbol{\phi}(x_t, a)^\top \mathbf{V}_t^{-1} \boldsymbol{\phi}(x_t, a)}, \tag{49}$$

where $\alpha > 0$ is an exploration parameter. The first term is the predicted reward; the second is the *confidence width*, equal to the Mahalanobis norm of $\boldsymbol{\phi}(x_t, a)$ under $\mathbf{V}_t^{-1}$. Contexts whose feature vectors lie in under-explored directions receive larger bonuses.

The agent selects $a_t = \arg\max_a \mathrm{UCB}_t(a)$, observes reward r_t, and updates $\mathbf{V}_{t+1}$ and $\mathbf{b}_{t+1}$.

Regret bound. Under standard assumptions (bounded features, sub-Gaussian noise), LinUCB achieves

$$\mathbb{E}[R_T] = O(d\sqrt{T \log T}). \tag{50}$$

This bound is optimal up to logarithmic factors for d-dimensional linear contextual bandits. The key advantage over non-contextual UCB1 is that regret scales with feature dimension d rather than arm count K—when $K \gg d$, this is a dramatic improvement.

6.4 Thompson Sampling for Contextual Bandits

Thompson Sampling extends naturally from the multi-armed bandit setting of Section 5 to the contextual case. Rather than maintaining independent Beta posteriors over each arm's success probability, the agent maintains a posterior distribution over $\boldsymbol{\theta}$ and samples from it at each round.

For linear models, assume:

$$r_t = \boldsymbol{\phi}(x_t, a_t)^\top \boldsymbol{\theta} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, \sigma^2), \quad \boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \sigma_\theta^2 \mathbf{I}). \tag{51}$$

The posterior over $\boldsymbol{\theta}$ remains Gaussian by conjugacy:

$$\boldsymbol{\theta} \mid \text{data} \sim \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t), \tag{52}$$

where

$$\boldsymbol{\Sigma}_t = \left(\sigma^{-2} \sum_{s=1}^{t} \boldsymbol{\phi}(x_s, a_s) \boldsymbol{\phi}(x_s, a_s)^\top + \sigma_\theta^{-2} \mathbf{I} \right)^{-1}, \tag{53}$$

$$\boldsymbol{\mu}_t = \sigma^{-2} \boldsymbol{\Sigma}_t \sum_{s=1}^{t} r_s \boldsymbol{\phi}(x_s, a_s). \tag{54}$$

At each round: (1) observe x_t, (2) sample $\tilde{\boldsymbol{\theta}} \sim \mathcal{N}(\boldsymbol{\mu}_{t-1}, \boldsymbol{\Sigma}_{t-1})$, (3) select $a_t = \arg\max_a \boldsymbol{\phi}(x_t, a)^\top \tilde{\boldsymbol{\theta}}$, and (4) update the posterior.

Regret bound. Thompson Sampling for linear contextual bandits achieves $\mathbb{E}[R_T] = O(d\sqrt{T \log T})$ [22], matching LinUCB. It often performs better empirically because exploration is proportional to uncertainty in a natural Bayesian sense, and it requires no tuning of a confidence parameter α.

6.5 EXP4: Adversarial Contextual Bandits

The stochastic setting assumes rewards are drawn from fixed distributions. In adversarial settings, an adversary may choose rewards after observing the algorithm's behavior. **EXP4** [13] addresses this by competing with a finite set of N *experts* (policies) $\pi_1, \ldots, \pi_N$, where each $\pi_i : \mathcal{X} \to \mathcal{A}$.

EXP4 maintains weights $w_{i,t}$ over experts. At each round:

1. Compute mixing probabilities: $p_t(a) = \sum_i w_{i,t} \mathbb{I}[\pi_i(x_t) = a] / \sum_j w_{j,t}$.
2. Sample $a_t \sim p_t(\cdot)$ and observe $r_t(a_t)$.
3. Construct importance-weighted estimates: $\hat{r}_t(a) = r_t(a)/p_t(a)$ if $a = a_t$, else 0.
4. Update weights: $w_{i,t+1} = w_{i,t} \exp(\eta \hat{r}_t(\pi_i(x_t)))$.

Regret bound. EXP4 achieves $\mathbb{E}[R_T] = O(\sqrt{TK \ln N})$ against the best expert in hindsight, with no distributional assumptions. This is the price of adversarial robustness: the bound depends on $\sqrt{K}$ rather than $\sqrt{d}$.

6.6 Example: News Article Recommendation

Consider a news website recommending articles to users. There are two user types (contexts): "tech-interested" and "sports-interested," and three articles (arms): Tech, Sports, and General. We use a 2-dimensional feature representation for tractable hand calculation.

Feature vectors:

Context–Arm Pair	$\boldsymbol{\phi}$	Expected Reward
Tech user, Tech article	$[1, 0]$	0.8
Tech user, Sports article	$[0, 0.5]$	0.2
Tech user, General article	$[0.5, 0.5]$	0.6
Sports user, Tech article	$[0, 0.5]$	0.2
Sports user, Sports article	$[1, 0]$	0.8
Sports user, General article	$[0.5, 0.5]$	0.6

The true parameter is $\boldsymbol{\theta}^* = [0.8, 0.4]$, so expected rewards are $\boldsymbol{\phi}^\top \boldsymbol{\theta}^*$. Tech users prefer Tech articles (0.8 vs 0.6 vs 0.2); Sports users prefer Sports articles (0.8 vs 0.6 vs 0.2).

LinUCB trace with $\alpha = 1$, $\lambda = 1$, alternating contexts, and noiseless rewards ($r_t = \boldsymbol{\phi}^\top \boldsymbol{\theta}^*$):

t	Context	$\hat{\boldsymbol{\theta}}_t$	UCB(Tech)	UCB(Sports)	UCB(Gen)	Choice
1	Sports	[0.00, 0.00]	0.50	1.00	0.71	Sports
2	Tech	[0.40, 0.00]	1.11	0.50	0.81	Tech
3	Sports	[0.53, 0.00]	0.50	1.11	0.85	Sports
4	Tech	[0.60, 0.00]	1.10	0.50	0.86	Tech
5	Sports	[0.64, 0.00]	0.50	1.09	0.87	Sports
6	Tech	[0.67, 0.00]	1.08	0.50	0.87	Tech

The feature structure enables LinUCB to identify the correct arm for each user type from the first round: the matching arm (Sports for Sports users, Tech for Tech users) has feature vector $[1, 0]$, which has the largest norm and thus the highest exploration bonus. As data accumulates, the first component of $\hat{\boldsymbol{\theta}}_t$ converges toward 0.8, and the algorithm consistently matches users with their preferred content. Note that $\hat{\boldsymbol{\theta}}_t$ converges to approximately $[0.8, 0]$ rather than the true $[0.8, 0.4]$: because the algorithm always selects arms whose features lie along $[1, 0]$, the second feature dimension is never explored in these early rounds. This illustrates a known property of linear bandits—the algorithm may not recover the full parameter vector, yet it still selects the correct arms. (After many more rounds, the persistent uncertainty in the second dimension eventually causes the General article's UCB to exceed the matching arm's, triggering exploration of the remaining parameter.)

The following Python code verifies this solution:

```
import numpy as np

d, K = 2, 3
alpha, lambda_reg = 1.0, 1.0
theta_star = np.array([0.8, 0.4])

def get_features(user, arm):
    feats = {(0,0): [1,0], (0,1): [0,0.5], (0,2): [0.5,0.5],
             (1,0): [0,0.5], (1,1): [1,0], (1,2): [0.5,0.5]}
    return np.array(feats[(user, arm)])

V = lambda_reg * np.eye(d)
b = np.zeros(d)
contexts = [1, 0, 1, 0, 1, 0] # Sports, Tech, alternating

# Trace: 6 rounds with noiseless rewards (reproduces table)
for t in range(6):
    user = contexts[t]
    theta_hat = np.linalg.solve(V, b)
    ucbs = [get_features(user, a) @ theta_hat +
            alpha * np.sqrt(get_features(user, a) @
            np.linalg.solve(V, get_features(user, a))) for a in range(K)]
    arm = np.argmax(ucbs)
    phi = get_features(user, arm)
    r = phi @ theta_star # noiseless for trace verification
    V += np.outer(phi, phi)
    b += r * phi
```

```
print(f"Estimated theta: {np.linalg.solve(V, b)}") # [0.80, 0.00]
```

Listing 3: LinUCB for the news recommendation example.

6.7 Summary

Contextual bandits extend the multi-armed bandit by incorporating side information that changes the optimal arm from round to round. The agent must learn a mapping from contexts to actions, generalizing across similar contexts rather than treating each arm independently. LinUCB and Thompson Sampling achieve $O(d\sqrt{T})$ regret under linear reward models, with regret scaling in feature dimension rather than arm count. EXP4 provides adversarial robustness at the cost of worse dependence on the action space. These algorithms underpin modern recommendation systems, personalized medicine, and online advertising.

6.7.1 Key Equations

Expected reward (linear model):

$$\mathbb{E}[r_t(a) \mid x_t] = \boldsymbol{\phi}(x_t, a)^\top \boldsymbol{\theta}^*.$$

Cumulative regret:

$$R_T = \sum_{t=1}^{T} \left[\max_a \boldsymbol{\phi}(x_t, a)^\top \boldsymbol{\theta}^* - \boldsymbol{\phi}(x_t, a_t)^\top \boldsymbol{\theta}^* \right].$$

LinUCB upper confidence bound:

$$\mathrm{UCB}_t(a) = \boldsymbol{\phi}(x_t, a)^\top \hat{\boldsymbol{\theta}}_t + \alpha \sqrt{\boldsymbol{\phi}(x_t, a)^\top \mathbf{V}_t^{-1} \boldsymbol{\phi}(x_t, a)}.$$

LinUCB / Thompson Sampling regret:

$$\mathbb{E}[R_T] = O(d\sqrt{T \log T}).$$

6.7.2 Relevant Applications

- News article and content recommendation, where user profile is the context and articles are arms.
- Personalized medicine and adaptive clinical trials, where patient features guide treatment selection.

- Online advertising and ad placement, where user demographics and browsing history inform bid strategies.
- Dynamic pricing, where product features and market conditions determine optimal price points.
- Dialog systems and chatbot response selection conditioned on conversation history.

7 Restless Bandit Problems

The classical multi-armed bandit of Section 5 assumes that *unplayed arms are frozen*: an arm's state changes only when it is pulled. In many applications this is unrealistic—machines degrade whether or not they are serviced, communication channels fluctuate whether or not they are probed, and patients' health evolves whether or not they receive treatment. The **restless bandit** model, introduced by Whittle [16], removes this restriction: every arm transitions at every time step, regardless of whether it is played. This seemingly modest generalization has profound consequences for tractability and policy design.

7.1 Notation

Symbol	Description
$\mathcal{S}_i$	State space of arm i
$p_{ij}^{(1)}$	Transition probability under the active action
$p_{ij}^{(0)}$	Transition probability under the passive action
$r_i(s)$	Reward of arm i in state s
M	Number of arms activated per time step
a_i^t	Activation indicator for arm i at time t
$\mathbf{P}^{(0)}$	Passive transition matrix
$\mathbf{P}^{(1)}$	Active transition matrix
$\mathbf{I}$	Identity matrix
λ	Lagrange multiplier (subsidy for passivity)
$W(s)$	Whittle index of state s
$V^{\text{Whittle}}(\mathbf{s})$	Value under the Whittle index policy at joint state $\mathbf{s}$

7.2 Formal Definition

A restless bandit consists of K independent arms, each modeled as a two-action MDP. Arm i has a finite state space $\mathcal{S}_i$ and two actions: *active* (play the arm) and *passive* (leave it alone).

The transition dynamics depend on the action taken:

$$\text{active:} \quad p_{ij}^{(1)} = P(s' = j \mid s = i, \text{active}), \tag{55}$$

$$\text{passive:} \quad p_{ij}^{(0)} = P(s' = j \mid s = i, \text{passive}). \tag{56}$$

The reward $r_i(s)$ depends on the state of arm i.[11] At each time step, the agent must activate exactly M of the K arms (with $M < K$); the remaining $K - M$ arms evolve passively. The objective is to maximize the expected total discounted reward:

$$\max \ \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \sum_{i=1}^{K} r_i(s_i^t)\right] \quad \text{subject to} \quad \sum_{i=1}^{K} a_i^t = M \ \text{ for all } t, \tag{57}$$

where $a_i^t \in \{0, 1\}$ indicates whether arm i is active at time t.

The classical ("rested") bandit of Section 5 is the special case where passive arms are frozen: $\mathbf{P}^{(0)} = \mathbf{I}$. In the restless model, $\mathbf{P}^{(0)} \neq \mathbf{I}$, so all arms evolve simultaneously.

7.3 Intractability

The joint state space of a K-arm restless bandit is the product $\mathcal{S}_1 \times \cdots \times \mathcal{S}_K$, which grows exponentially in K. Papadimitriou and Tsitsiklis [17] proved that the restless bandit problem is **PSPACE-hard**—a strictly stronger form of intractability than NP-hardness. This motivates the use of index heuristics.

7.4 Whittle's Relaxation and the Whittle Index

Whittle [16] proposed a Lagrangian relaxation that replaces the hard constraint (activate exactly M arms at every step) with a time-average constraint:

$$(1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \sum_{i=1}^{K} \mathbb{E}[a_i^t] \leq M. \tag{58}$$

Introducing a Lagrange multiplier λ—interpreted as a *subsidy for passivity*—the relaxed objective becomes

$$\max \ \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \sum_{i=1}^{K} \left(r_i(s_i^t) + \lambda\,(1 - a_i^t)\right)\right]. \tag{59}$$

This decomposes into K independent single-arm subproblems: for each arm, solve a two-action MDP with an additional reward of λ for choosing the passive action. Each subproblem is a standard MDP and can be solved in polynomial time.

[11] More generally, the reward may also depend on the action: $r_i(s, a)$. We use $r_i(s)$ here for simplicity, as in many applications the reward reflects the arm's state rather than the chosen action.

An arm is said to be **indexable** if the set of states in which the passive action is optimal grows monotonically as λ increases—that is, as passivity becomes more lucrative, the agent becomes passive in progressively more states. When indexability holds, the **Whittle index** of a state s is the critical subsidy

$$W(s) = \inf\{\lambda : \text{passive is optimal in state } s \text{ under subsidy } \lambda\}. \tag{60}$$

Intuitively, $W(s)$ measures the *value of activating* an arm in state s: it is the minimum per-period payment required to make the agent willing to leave that arm idle.

The **Whittle index policy** is then: at each time step, activate the M arms with the highest Whittle indices. This policy is a heuristic—it is not guaranteed to be optimal—but Weber and Weiss [18] proved that it is **asymptotically optimal** as $K \to \infty$ (with M/K held constant), provided the arms are indexable and a technical stability condition holds.

Not all restless bandits are indexable. When indexability fails, the Whittle index is not well-defined and alternative approaches (LP relaxations, reinforcement learning) must be used. Verifying indexability for a given arm requires checking that its optimal policy has a threshold structure in λ.

7.5 Example: Machine Maintenance

Consider $K = 3$ identical machines and $M = 1$ repairman. Each machine has states $\mathcal{S} = \{\text{G (Good)}, \text{B (Bad)}\}$ with rewards $r(\text{G}) = 1$ and $r(\text{B}) = 0$. The transition matrices are:

$$\mathbf{P}^{(0)} = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}, \qquad \mathbf{P}^{(1)} = \begin{pmatrix} 0.95 & 0.05 \\ 0.7 & 0.3 \end{pmatrix}, \tag{61}$$

where rows correspond to the current state (G, B) and columns to the next state. Under passive operation, a Good machine degrades with probability 0.1 per period; a Bad machine self-repairs with probability 0.2. Active maintenance (the repairman) reduces degradation and greatly improves repair: a Bad machine returns to Good with probability 0.7 instead of 0.2. Figure 7 shows these transitions.

The discount factor is $\gamma = 0.9$. Computing the Whittle indices by binary search on λ yields:

$$W(\text{G}) \approx 0.058, \qquad W(\text{B}) \approx 1.216. \tag{62}$$

The large gap $W(\text{B}) \gg W(\text{G})$ reflects an intuitive priority: a broken machine benefits far more from the repairman than a functioning one.

The Whittle index policy assigns the repairman to the machine with the highest index—always a Bad machine when one exists. We trace five steps starting from states $(\text{G}, \text{B}, \text{B})$, where M1, M2, M3 denote machines 1, 2, and 3 respectively:

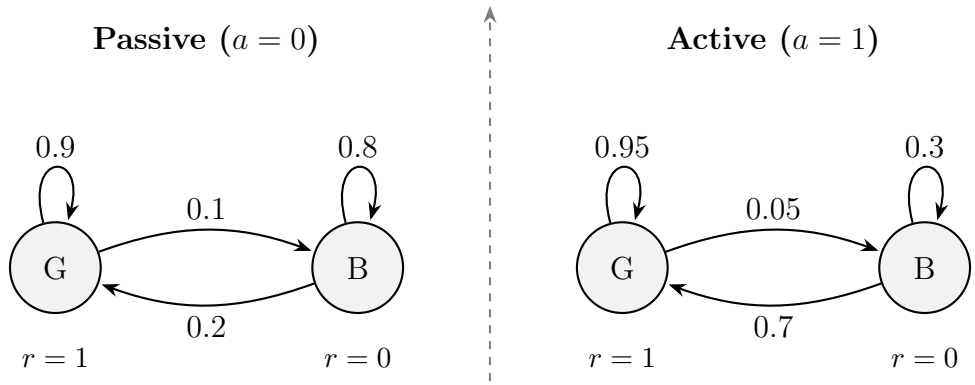

Figure 7: Single-arm transitions for the machine maintenance restless bandit. Left: passive (unattended) transitions. Right: active (repairman assigned) transitions. The key difference is in the Bad→Good probability (0.2 vs. 0.7). Unlike classical bandits, the passive transitions are *not* frozen—the machine degrades whether or not the repairman is present.

t	States	W	Activate	Transitions	Reward
0	(G, B, B)	(0.06, **1.22**, 1.22)	M2	M2: B→G	1
1	(G, G, B)	(0.06, 0.06, **1.22**)	M3	M3: B→G	2
2	(G, G, G)	(0.06, 0.06, 0.06)	M1	M2: G→B	3
3	(G, B, G)	(0.06, **1.22**, 0.06)	M2	M3: G→B	2
4	(G, G, B)	(0.06, 0.06, **1.22**)	M3	M3: B→G	2

Several features of restless bandits are visible in this trace. At $t = 0$, machines 2 and 3 are both Bad; the repairman services machine 2 (tie-breaking), and it returns to Good. Meanwhile machine 3 remains passive and stays Bad—it does not freeze as it would in a classical bandit. At $t = 2$, all machines are Good and the indices are equal; the repairman performs preventive maintenance on machine 1, but machine 2 degrades (G→B with passive probability 0.1) despite being unattended. This *degradation while idle* is the defining feature of the restless model.

For this small instance, the Whittle index policy can be verified against the brute-force optimum over all $2^3 = 8$ joint states. The two policies differ at three states (e.g., in state $(\mathrm{B}, \mathrm{B}, \mathrm{B})$, the optimal policy may activate machine 3 while the Whittle policy activates machine 1), but because the machines are identical these tie-breaking differences have no effect on value: $V^*(\mathbf{s}) = V^{\text{Whittle}}(\mathbf{s})$ for all $\mathbf{s}$.

The following Python code computes the Whittle indices and solves the exact problem via value iteration:

```
import numpy as np
from itertools import product

gamma = 0.9
P0 = np.array([[0.9, 0.1], [0.2, 0.8]]) # passive
P1 = np.array([[0.95, 0.05], [0.7, 0.3]]) # active
```

```
r = np.array([1.0, 0.0]) # r(G)=1, r(B)=0

# --- Whittle index via binary search on subsidy lambda ---
for state in [0, 1]: # 0=Good, 1=Bad
    lo, hi = -10.0, 10.0
    for _in range(200):
        lam = (lo + hi) / 2.0
        V = np.zeros(2)
        for _in range(100000):
            Qp = r + lam + gamma * P0 @ V
            Qa = r + gamma * P1 @ V
            V_new = np.maximum(Qp, Qa)
            if np.max(np.abs(V_new - V)) < 1e-12:
                break
            V = V_new
        if Qp[state] > Qa[state]:
            hi = lam
        else:
            lo = lam
    name = "Good" if state == 0 else "Bad"
    print(f"W({name}) = {(lo+hi)/2:.4f}")

# --- Brute-force optimal for K=3, M=1 ---
states = list(product([0, 1], repeat=3))
V = np.zeros(8)
for _in range(100000):
    V_new = np.zeros(8)
    for si, s in enumerate(states):
        best = -float("inf")
        for a in range(3): # activate machine a
            rew = sum(r[s[i]] for i in range(3))
            P = [P1 if i == a else P0 for i in range(3)]
            val = rew + gamma * sum(
                np.prod([P[i][s[i]][sn[i]] for i in range(3)])
                * V[sn[0]*4 + sn[1]*2 + sn[2]]
                for sn in product([0,1], repeat=3))
            best = max(best, val)
        V_new[si] = best
    if np.max(np.abs(V_new - V)) < 1e-12:
        break
    V = V_new
for si, s in enumerate(states):
    names = ["G" if x == 0 else "B" for x in s]
    print(f"V*({','.join(names)}) = {V[si]:.2f}")
```

Listing 4: Whittle index computation and verification for the machine maintenance restless bandit.

7.6 Summary

The restless bandit generalizes the classical multi-armed bandit by allowing all arms to evolve at every time step, whether or not they are played. This removes the "frozen when idle" assumption of the Gittins model and greatly increases the problem's complexity—from polynomial-time solvable (via the Gittins index) to PSPACE-hard. Whittle's Lagrangian relaxation partially restores tractability by decomposing the coupled problem into independent single-arm subproblems, yielding the Whittle index policy. This heuristic is asymptotically optimal under indexability and stability conditions, and performs well in practice across a range of domains.

7.6.1 Key Equations

Restless arm transition dynamics (active and passive):

$$p_{ij}^{(1)} = P(s' = j \mid s = i, \text{active}), \qquad p_{ij}^{(0)} = P(s' = j \mid s = i, \text{passive}).$$

Whittle's Lagrangian relaxation (time-average activation constraint):

$$(1-\gamma)\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{K}\mathbb{E}[a_i^t] \le M.$$

Relaxed objective with subsidy for passivity:

$$\max\ \mathbb{E}\left[\sum_{t=0}^{\infty}\gamma^t\sum_{i=1}^{K}\left(r_i(s_i^t) + \lambda\,(1-a_i^t)\right)\right].$$

Whittle index (critical subsidy):

$$W(s) = \inf\big\{\lambda : \text{passive is optimal in state } s \text{ under subsidy } \lambda\big\}.$$

7.6.2 Relevant Applications

- Preventive maintenance scheduling for fleets of machines or infrastructure assets that degrade whether or not they are serviced.
- Wireless channel scheduling, where channel quality fluctuates due to fading and interference regardless of whether a channel is used.
- Healthcare intervention programs (e.g., maternal health, chronic disease management), where patient adherence and health status evolve between contacts.

- Sensor management and target tracking, where targets move and hide whether or not they are under surveillance.
- Web crawling and content caching, where page freshness decays whether or not pages are re-fetched.

8 Beyond the Scope of This Paper

The preceding sections trace a single thread through the landscape of Markov models: passive chains (Section 2), controlled chains with full observability (Section 3), controlled chains with partial observability (Section 4), stateless exploration–exploitation (Section 5), exploration with side information (Section 6), and the restless generalization where arms evolve independently (Section 7). This thread is coherent but not exhaustive. Several major model classes occupy adjacent or intersecting regions of the same landscape and are briefly noted here.

8.1 Hidden Markov Models

A **hidden Markov model** (HMM) [19] is a Markov chain whose states are unobservable; at each time step the system emits an observation drawn from a state-dependent distribution. The HMM is the passive, uncontrolled counterpart of the POMDP: it shares the hidden-state and observation machinery of Section 4 but has no actions or rewards. This completes a natural 2×2 classification—{observable, hidden} $\times$ {passive, controlled} = {Markov chain, HMM, MDP, POMDP}. The central computational problems for HMMs—filtering (the forward algorithm), smoothing (the forward–backward algorithm), and parameter estimation (Baum–Welch / EM)—underpin speech recognition, gene finding, and financial time-series analysis.

Applications:

- Speech recognition, where hidden phoneme sequences generate observed acoustic signals.
- Gene finding and biological sequence analysis, where hidden functional regions emit nucleotide patterns.
- Financial regime detection, where latent market states (bull, bear, crisis) drive observable returns.
- Part-of-speech tagging and natural language processing, where hidden grammatical roles produce observed words.

- Activity recognition from sensor data, where unobserved human activities generate accelerometer and gyroscope readings.

8.2 Markov Games

All models in Sections 2–7 involve a single decision-maker. A **Markov game** (or stochastic game) [23] generalizes the MDP to multiple agents who act simultaneously, each with their own reward function. Transition probabilities and rewards depend on the *joint* action of all agents. The solution concept shifts from an optimal policy to a Nash equilibrium—a profile of policies from which no agent can unilaterally improve. Special cases include zero-sum games (purely competitive) and team games (fully cooperative). When agents also have private observations, the model becomes a **decentralized POMDP** (Dec-POMDP) [24], which is NEXP-complete—harder even than single-agent POMDPs.

Applications:

- Multi-player games and competitive AI (poker, real-time strategy, autonomous racing).
- Multi-robot coordination for search-and-rescue, warehouse logistics, or formation control.
- Economic market modeling, where firms or agents interact strategically over time.
- Cybersecurity, where attacker and defender adapt strategies in response to each other.
- Multi-agent traffic signal control in urban networks.

8.3 Constrained MDPs

The MDP of Section 3 optimizes a single scalar objective. A **constrained MDP** (CMDP) [25] adds side constraints of the form $\mathbb{E}[\sum_t \gamma^t c_k(s_t, a_t)] \leq C_k$, requiring that expected cumulative costs remain within budgets. The optimal policy is generally a *mixture* of deterministic policies, computable via linear programming. CMDPs are increasingly important in safety-critical applications—autonomous driving, healthcare, robotics—where constraint satisfaction (e.g., collision probability, dosage limits) is as important as reward maximization.

Applications:

- Autonomous driving with safety constraints on collision probability and lane departure.
- Clinical trial design with dosage limits and adverse-event budgets.
- Power grid management subject to emission caps and reliability standards.

- Portfolio optimization with drawdown or value-at-risk constraints.
- Wireless resource allocation under fairness or quality-of-service guarantees.

8.4 Semi-Markov Decision Processes

The MDP assumes that every action takes exactly one time step. A **semi-Markov decision process** (SMDP) [26] relaxes this by allowing actions to have variable, random durations (sojourn times). Rewards accrue over the holding period, and discounting accounts for elapsed time rather than step count. SMDPs are natural models for manufacturing, maintenance scheduling, and queueing, where the time between decisions varies. They also provide the formal foundation for the *options* framework in hierarchical reinforcement learning, where temporally extended actions (options) execute over multiple primitive steps.

Applications:

- Maintenance scheduling for industrial equipment, where repair and inspection durations vary.
- Queueing systems and call-center staffing, where service times are random.
- Inventory management with stochastic lead times for replenishment.
- Hierarchical task planning in robotics, where high-level skills (options) span variable numbers of primitive actions.
- Telecommunications network routing, where packet travel times fluctuate with congestion.

8.5 Model-Free Reinforcement Learning

The solution methods in this paper—value iteration, policy evaluation, Bellman equations solved in closed form—all assume the transition probabilities and rewards are known. **Model-free reinforcement learning** [27] addresses the setting where the agent must learn to act *without* a model, using only sampled experience. Algorithms such as Q-learning, SARSA, and policy gradient methods (REINFORCE, PPO, actor-critic) learn value functions or policies directly from interaction. Deep reinforcement learning combines these algorithms with neural network function approximation, enabling application to problems with enormous state spaces—game playing, robotic manipulation, chip design—where tabular dynamic programming is infeasible. Model-free RL is not a model class in the same sense as the others listed here; it is a solution methodology that applies across MDPs, POMDPs, and multi-agent settings.

Applications:

- Game playing (Atari, Go, chess, poker) where the state space is too large for tabular methods.
- Robotic locomotion and manipulation, where policies are learned from simulated or real-world interaction.
- Autonomous vehicle control, including lane keeping, merging, and racing.
- Chip placement and hardware design optimization.
- Large language model alignment via reinforcement learning from human feedback (RLHF).

9 Conclusion

This paper has traced a single coherent thread through the theory of sequential decision-making under uncertainty: from the passive Markov chain (Section 2), where a system evolves according to fixed transition probabilities, to the Markov decision process (Section 3), where an agent's actions influence both transitions and rewards, to the partially observable MDP (Section 4), where the agent must act on incomplete information maintained through Bayesian belief updates. The multi-armed bandit (Section 5) isolates the exploration–exploitation tradeoff in its purest form; the contextual bandit (Section 6) reintroduces structure by conditioning arm rewards on observed side information; and the restless bandit (Section 7) allows arms to evolve whether or not they are played.

Several unifying ideas recur across these models. The *Bellman equation*—the principle that an optimal policy decomposes into an optimal immediate action followed by optimal continuation—appears in Sections 3, 4, and (implicitly, via the Gittins index) Section 5. *Sufficient statistics* play a central role: the state in an MDP, the belief in a POMDP, and the posterior in a Bayesian bandit each compress all relevant history into a compact representation. The tension between *tractability and generality* is a recurring theme: each extension in the hierarchy (adding actions, partial observability, or restless dynamics) increases modeling power at the cost of computational complexity, from polynomial-time solvability for MDPs to PSPACE-hardness for POMDPs and restless bandits.

The worked examples throughout the paper—the two-state Markov chain, the binary MDP and asset maintenance problem, the Tiger Problem, the three-armed bandit under four strategies, the news recommendation contextual bandit, and the machine maintenance restless bandit—are intended to make the formal definitions concrete. Each example is accompanied by explicit numerical computation and, where appropriate, Python code for verification. Our hope is that these examples lower the barrier to entry for practitioners encountering these models for the first time.

Section 8 surveyed several important model classes that lie outside the scope of this paper, including hidden Markov models, Markov games, constrained MDPs, semi-Markov decision

processes, and model-free reinforcement learning. Together with the models treated here, they span much of the landscape of Markov-based sequential decision-making. Exploring these extensions—and the connections among them—remains a rich area of both theoretical research and practical application. These models collectively form the mathematical foundation for modern applications ranging from autonomous systems and healthcare decision-making to recommendation engines and game-playing artificial intelligence.

Glossary

Action A decision available to the agent at each time step. In an MDP, the set of actions is $\mathcal{A} = \{a_1, \ldots, a_m\}$. (Section 3)

Asymptotic optimality A property of a policy whose performance approaches the optimum as a problem parameter (e.g., K or T) tends to infinity. The Whittle index policy is asymptotically optimal under indexability and stability conditions. (Section 7)

Belief MDP The reformulation of a POMDP as a fully observable MDP whose state is the belief vector $\mathbf{b}$. The Bellman equation carries over but ranges over a continuous state space. (Section 4)

Belief state A probability distribution $\mathbf{b}$ over the state space $\mathcal{S}$, representing the agent's uncertainty about the true state in a POMDP. Updated via Bayes' rule after each action and observation. (Section 4)

Bellman equation The recursive characterization of the optimal value function: $V^*(s_i) = \max_a[r(s_i, a) + \gamma \sum_j p_{ij}(a)V^*(s_j)]$. Expresses the principle of optimality. (Section 3)

Constrained MDP (CMDP) An MDP augmented with side constraints on expected cumulative costs. The optimal policy is generally a mixture of deterministic policies. (Section 8)

Contextual bandit A bandit model in which the agent observes a context (feature vector) before selecting an arm, and the reward distribution depends on the context. The agent must learn a mapping from contexts to optimal arms. Intermediate between the context-free bandit and the full MDP. (Section 6)

Cumulative regret The total expected reward lost by not always pulling the best arm: $R_T = \sum_{i:\mu_i<\mu^*} \Delta_i \cdot \mathbb{E}[T_i(T)]$. (Section 5)

Discount factor A parameter $\gamma \in [0, 1)$ that weights future rewards relative to immediate ones. A reward received t steps in the future is worth γ^t times its face value. (Section 3)

Ergodicity A Markov chain is ergodic if it is irreducible (every state is reachable from every other) and aperiodic (does not cycle with a fixed period). An ergodic chain converges to a unique stationary distribution. (Section 2)

Exploration The act of trying actions or arms to gather information about their rewards or transition dynamics, at the cost of foregoing the currently best-known option. Contrasted with exploitation. (Section 5)

Exploitation The act of choosing the action or arm currently believed to be best, based on available information. Contrasted with exploration. (Section 5)

EXP3 Exponential-weight algorithm for Exploration and Exploitation. A bandit strategy for the adversarial (non-stochastic) setting, achieving $O(\sqrt{KT \ln K})$ regret. (Section 5)

Gap The suboptimality of arm i: $\Delta_i = \mu^* - \mu_i$. Arms with smaller gaps are harder to distinguish from the optimum. (Section 5)

Gittins index The index G_i assigned to each arm in the discounted Bayesian bandit, computed from the arm's posterior alone. The Bayes-optimal policy pulls the arm with the highest Gittins index. (Section 5)

Hidden Markov model (HMM) A Markov chain with unobservable states and state-dependent observations. The passive, uncontrolled counterpart of the POMDP. (Section 8)

Immediate reward The reward $r(s_i, a)$ received when action a is taken in state s_i. (Section 3)

Indexability A property of a restless bandit arm: the set of states in which the passive action is optimal grows monotonically as the subsidy for passivity λ increases. Required for the Whittle index to be well-defined. (Section 7)

KL-UCB A variant of UCB that uses Kullback–Leibler divergence instead of Hoeffding bounds, achieving asymptotically optimal regret with exact constants. (Section 5)

Lai–Robbins lower bound The information-theoretic lower bound on regret: $\liminf_{T\to\infty} \mathbb{E}[R_T]/\ln T \geq \sum_i \Delta_i/\mathrm{KL}(\nu_i, \nu^*)$. Establishes that logarithmic regret growth is the best achievable. (Section 5)

LinUCB Linear Upper Confidence Bound. A contextual bandit algorithm that models expected rewards as linear in context features and uses ridge regression to estimate parameters with confidence bounds. Achieves $O(d\sqrt{T})$ regret where d is the feature dimension. (Section 6)

Markov chain See Markov process. (Section 2)

Markov decision process (MDP) A controlled stochastic process defined by $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$, extending the Markov chain with actions and rewards. (Section 3)

Markov game A multi-agent extension of the MDP in which multiple players choose actions simultaneously and each receives a potentially different reward. Also called a stochastic game. (Section 8)

Markov process A stochastic process satisfying the Markov property: the future state depends only on the present state, not on the history. Also called a Markov chain. (Section 2)

Markov property The memoryless property: $P(X_{t+1} = s_j \mid X_t = s_i, X_{t-1}, \ldots, X_0) = P(X_{t+1} = s_j \mid X_t = s_i)$. (Section 2)

Model-free reinforcement learning Solution methods (e.g., Q-learning, policy gradient) that learn to act without a model of transition probabilities or rewards, using only sampled experience. (Section 8)

Multi-armed bandit (MAB) A sequential decision problem in which an agent repeatedly chooses among K arms with unknown reward distributions, balancing exploration and exploitation. A degenerate MDP with a single state. (Section 5)

Observation function The function $O(o \mid s', a)$ giving the probability of receiving observation o after taking action a and arriving in state s'. (Section 4)

Optimal policy The policy π^* that achieves the maximum value at every state: $\pi^*(s_i) = \arg\max_a [r(s_i, a) + \gamma \sum_j p_{ij}(a) V^*(s_j)]$. (Section 3)

Optimal value function The function $V^*(s_i) = \max_\pi V^\pi(s_i)$ giving the maximum achievable discounted return from each state. (Section 3)

Partially observable MDP (POMDP) An MDP in which the agent cannot observe the true state and instead receives noisy observations. Defined by $(\mathcal{S}, \mathcal{A}, p, r, \Omega, O, \gamma)$. (Section 4)

Policy A rule $\pi : \mathcal{S} \to \mathcal{A}$ specifying which action to take in each state. Under a fixed policy, an MDP reduces to a Markov chain. (Section 3)

PSPACE-hard A complexity class indicating that a problem is at least as hard as any problem solvable in polynomial space. Exact finite-horizon POMDP planning and the restless bandit problem are both PSPACE-hard. (Sections 4, 7)

Restless bandit A multi-armed bandit in which every arm transitions at every time step, regardless of whether it is played. Introduced by Whittle (1988). (Section 7)

Semi-Markov decision process (SMDP) An MDP in which actions have variable, random durations. Rewards accrue over the holding period and discounting accounts for elapsed time. (Section 8)

Stationary distribution A distribution $\boldsymbol{\mu}^*$ satisfying $\boldsymbol{\mu}^* = \boldsymbol{\mu}^* \mathbf{P}$; once reached, it remains unchanged under further transitions. (Section 2)

Successive elimination A bandit algorithm that maintains a set of active arms and permanently discards arms whose confidence bounds prove them suboptimal. (Section 5)

Sufficient statistic A quantity that captures all information from the history relevant to future decisions. The state in an MDP, the belief in a POMDP, and the posterior in a Bayesian bandit are each sufficient statistics. (Sections 2–5)

Thompson Sampling A Bayesian bandit strategy that samples from the posterior distribution of each arm's mean and pulls the arm with the highest sample. (Section 5)

Transition matrix The matrix $\mathbf{P}$ with entries p_{ij}, where each row sums to one. Governs the evolution of the state distribution: $\boldsymbol{\mu}_{t+1} = \boldsymbol{\mu}_t \mathbf{P}$. (Section 2)

Transition probability The probability p_{ij} (or $p_{ij}(a)$ in an MDP) of moving from state s_i to state s_j (under action a). (Sections 2, 3)

UCB1 Upper Confidence Bound algorithm. Selects the arm maximizing $\hat{\mu}_i + \sqrt{2 \ln t / N_i}$, balancing exploitation (empirical mean) with exploration (confidence bonus). (Section 5)

Value function A function $V : \mathcal{S} \to \mathbb{R}$ assigning to each state the total discounted reward obtainable under a given policy (or the optimal policy). (Section 3)

Value iteration An algorithm that repeatedly applies the Bellman operator to converge to the optimal value function. (Section 3)

Whittle index The critical subsidy $W(s) = \inf\{\lambda : \text{passive is optimal in state } s\}$. Measures the value of activating a restless bandit arm in state s. (Section 7)

Whittle index policy The heuristic that activates the M arms with the highest Whittle indices at each time step. Asymptotically optimal under indexability and stability. (Section 7)

References

[1] А. А. Марков, «Распространение закона больших чисел на величины, зависящие друг от друга», *Изв. физ.-матем. общ. Казан. ун-та*, 2-я сер., т. 15, с. 135–156, 1906. [A. A. Markov, "Extension of the law of large numbers to dependent quantities," *Izvestiia Fiz.-Matem. Obshch. Kazan. Univ.*, 2nd Ser., vol. 15, pp. 135–156, 1906.]

[2] R. Bellman, "A Markovian decision process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.

[3] K. J. Åström, "Optimal control of Markov processes with incomplete state information I," *Journal of Mathematical Analysis and Applications*, vol. 10, pp. 174–205, 1965.

[4] R. D. Smallwood and E. J. Sondik, "The optimal control of partially observable Markov processes over a finite horizon," *Operations Research*, vol. 21, no. 5, pp. 1071–1088, 1973.

[5] C. H. Papadimitriou and J. N. Tsitsiklis, "The complexity of Markov decision processes," *Mathematics of Operations Research*, vol. 12, no. 3, pp. 441–450, 1987.

[6] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, nos. 1–2, pp. 99–134, 1998.

[7] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, nos. 3–4, pp. 285–294, 1933.

[8] H. Robbins, "Some aspects of the sequential design of experiments," *Bulletin of the American Mathematical Society*, vol. 58, pp. 527–535, 1952.

[9] J. C. Gittins, "Bandit processes and dynamic allocation indices," *Journal of the Royal Statistical Society: Series B*, vol. 41, no. 2, pp. 148–164 [with discussion, pp. 165–177], 1979.

[10] T. L. Lai and H. Robbins, "Asymptotically efficient adaptive allocation rules," *Advances in Applied Mathematics*, vol. 6, pp. 4–22, 1985.

[11] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, pp. 235–256, 2002.

[12] S. Agrawal and N. Goyal, "Analysis of Thompson Sampling for the multi-armed bandit problem," in *Proceedings of the 25th Annual Conference on Learning Theory (COLT)*, pp. 39.1–39.26, 2012.

[13] P. Auer, N. Cesa-Bianchi, Y. Freund, and R. E. Schapire, "The nonstochastic multi-armed bandit problem," *SIAM Journal on Computing*, vol. 32, no. 1, pp. 48–77, 2002.

[14] E. Even-Dar, S. Mannor, and Y. Mansour, "Action elimination and stopping conditions for the multi-armed bandit and reinforcement learning problems," *Journal of Machine Learning Research*, vol. 7, pp. 1079–1105, 2006.

[15] A. Garivier and O. Cappé, "The KL-UCB algorithm for bounded stochastic bandits and beyond," in *Proceedings of the 24th Annual Conference on Learning Theory (COLT)*, pp. 359–376, 2011.

[16] P. Whittle, "Restless bandits: activity allocation in a changing world," *Journal of Applied Probability*, vol. 25A, pp. 287–298, 1988.

[17] C. H. Papadimitriou and J. N. Tsitsiklis, "The complexity of optimal queuing network control," *Mathematics of Operations Research*, vol. 24, no. 2, pp. 293–305, 1999.

[18] R. R. Weber and G. Weiss, "On an index policy for restless bandits," *Journal of Applied Probability*, vol. 27, no. 3, pp. 637–648, 1990.

[19] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[20] J. Langford and T. Zhang, "The Epoch-Greedy algorithm for contextual multi-armed bandits," in *Advances in Neural Information Processing Systems 20 (NeurIPS)*, pp. 817–824, 2007.

[21] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pp. 661–670, 2010.

[22] S. Agrawal and N. Goyal, "Thompson Sampling for contextual bandits with linear payoffs," in *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pp. 127–135, 2013.

[23] L. S. Shapley, "Stochastic games," *Proceedings of the National Academy of Sciences*, vol. 39, no. 10, pp. 1095–1100, 1953.

[24] D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein, "The complexity of decentralized control of Markov decision processes," *Mathematics of Operations Research*, vol. 27, no. 4, pp. 819–840, 2002.

[25] E. Altman, *Constrained Markov Decision Processes*. Boca Raton, FL: Chapman & Hall/CRC, 1999.

[26] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York: Wiley, 1994.

[27] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[28] R. A. Howard, *Dynamic Programming and Markov Processes*. Cambridge, MA: MIT Press, 1960.

[29] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge: Cambridge University Press, 2020.